

OS161: Comprehensive Study Guide

Nicola Beccaceci

June 14, 2026

Contents

1	LAB 01: Introduction to OS161 and the Build System	4
1.1	The Nature of OS161 and Sys161	4
1.1.1	What is an Emulator?	4
1.1.2	Cross-Compilation Explained	5
1.2	Project Structure Overview	5
1.3	The OS161 Build System	5
1.3.1	1. Configuring the Kernel (<code>./config</code>)	5
1.3.2	2. Building the Dependency Graph (<code>bmake depend</code>)	6
1.3.3	3. Compilation and Linking (<code>bmake</code>)	6
1.3.4	4. Installation (<code>bmake install</code>)	6
1.3.5	VSCode Build and Debug Integration	6
1.4	Adding a Custom Boot Message (Configuration Deep Dive)	6
1.5	Distributed Kernel Debugging	8
1.6	Core OS161 Data Types: <code>vaddr_t</code> vs <code>paddr_t</code>	8
1.7	Introduction to OS161 Concurrency	9
1.7.1	Processes vs Threads	9
1.7.2	The Thread API	9
1.7.3	Testing and Debugging Threads	10
1.8	Laboratory 01: Print hello world in OS161	11
1.8.1	High-Level Overview	11
1.8.2	Step 1: Define the Option in <code>conf.kern</code>	11
1.8.3	Step 2: Enable the Option in DUMBVM	11
1.8.4	Step 3: Create the Source Code and Header File	12
1.8.5	Step 4: Integrate into the Boot Sequence	12
1.8.6	Step 5: Reconfigure the Build System	12
1.8.7	Step 6: Compile and Install	13
1.8.8	Step 7: Boot the Simulator	13
2	LAB 02: Running User Programs and System Calls	18
2.1	The Need for System Calls	18
2.2	The Trap Mechanism: Trapframe vs Switchframe	18
2.2.1	The Trapframe	18
2.2.2	The Switchframe	19
2.3	Process Execution: From Menu to User Mode	20
2.3.1	Deep Dive: <code>load_elf</code> and Executable Loading	20
2.4	The Syscall Dispatcher	22
2.5	Implementing File I/O and Memory Protection	22
2.5.1	Visualizing Logical Memory and The Danger of <code>kseg0</code>	22
2.5.2	The Solution: <code>copyin</code> and <code>copyout</code>	23
2.6	Process Termination Constraints: The Suicide Paradox	25
2.7	Memory Management: Upgrading DumbVM	27

2.7.1	The Flaw of <code>ram_stealmem</code>	27
2.7.2	Proposed Solution 1: Bitmap Allocation	28
2.7.3	Proposed Solution 2: Free-List Allocation	28
2.7.4	Address Space Duplication (<code>as_copy</code>)	29
2.7.5	The User Stack Allocation Mechanism (<code>getppages</code>)	29
2.7.6	MIPS Memory Map: User vs Kernel Space	30
2.7.7	Logical to Physical Address Translation in OS161	31
2.8	Laboratory 02: Execute a user program in OS161 (Step-by-Step)	32
2.8.1	The Goal and The Current Problem	32
2.8.2	Step 1: Implementing the System Calls	32
2.8.3	Step 2: Declaring the Syscall Prototypes	33
2.8.4	Step 3: Updating the Syscall Dispatcher	33
2.8.5	Step 4: Upgrading the Memory Manager	34
2.8.6	Step 5: Wiring the New Files into the Build System	35
2.8.7	Step 6: Reconfigure and Compile	35
2.8.8	Step 7: Testing the Execution	35
3	LAB 03: Synchronization in OS161	43
3.1	Introduction to Synchronization	43
3.2	Hardware Support: Disabling Interrupts vs. Spinlocks	43
3.2.1	Software Solutions: Peterson's Algorithm and Limitations	43
3.2.2	The Limitation of Disabling Interrupts	44
3.2.3	Spinlocks and the Test-and-Set Instruction	44
3.2.4	Advanced Hardware Instructions: Compare-and-Swap (CAS)	45
3.3	The Sleeping Engine: Wait Channels (<code>wchan</code>)	46
3.3.1	The Spinlock Sleep Deadlock	46
3.4	Semaphores vs. Locks	46
3.4.1	Semaphores	47
3.4.2	Locks and Ownership	48
3.5	Condition Variables and Mesa Semantics	49
3.5.1	The Necessity of the <code>while</code> Loop (Mesa Semantics)	51
3.5.2	CVs vs Wait Channels: The Chain of Calls	51
3.6	The Producer-Consumer Architecture	52
3.6.1	Architectural Placement of Shared Data	52
3.6.2	Implementation using Locks and CVs	53
3.7	Laboratory 03: Step-by-Step Guide	55
3.7.1	A Note on <code>KASSERT()</code>	55
3.7.2	The Goal and the Current Problem	55
3.7.3	Step 1: Upgrading the Lock Structure	55
3.7.4	Step 2: Implementing Lock Creation and Acquisition	55
3.7.5	Step 3: Implementing Lock Release and Ownership Validation	56
3.7.6	Step 4: Upgrading the Condition Variable	56
3.7.7	Step 5: Implementing <code>cv_wait</code> and Signals	57
3.7.8	Step 6: Enabling <code>OPT_SYNCH</code> and Testing	58
4	LAB 04: Process Termination and WaitPID	65
4.1	Introduction: The Interconnected Process Lifecycle	65
4.2	The Process Table and PID Assignment	65
4.2.1	High-Level Structure: The Process Table	67
4.2.2	Circular Buffer for PID Assignment	67
4.3	The Exit System Call and the Zombie State	67
4.3.1	The Zombie State	68

4.3.2	The Thread Termination Rule and the Race Condition	69
4.4	The Wait System Call: <code>sys_waitpid</code> vs <code>proc_wait</code>	70
4.4.1	User Layer: <code>sys_waitpid</code>	70
4.4.2	Kernel Layer: <code>proc_wait</code>	70
4.5	Applying Wait Logic to the Kernel Menu	71
4.6	Laboratory 04: Step-by-Step Guide	72
4.6.1	The Goal and the Current Problem	72
4.6.2	Step 1: Upgrading the Process Structure	72
4.6.3	Step 2: Defining the Process Table	72
4.6.4	Step 3: Circular PID Assignment	73
4.6.5	Step 4: Implementing <code>sys_exit</code>	73
4.6.6	Step 5: Implementing <code>proc_wait</code>	74
4.6.7	Step 6: Table Lookup and Cleanup Functions	74
4.6.8	Step 7: Enabling the WAITPID Option	75
4.6.9	Step 8: Testing the Execution	75
4.7	Global Architecture: The Process Termination Call Graph	76
4.8	The Open File Table (Preview)	78

Chapter 1

LAB 01: Introduction to OS161 and the Build System

1.1 The Nature of OS161 and Sys161

Operating systems are inherently complex software constructs, often spanning millions of lines of code, making them overwhelmingly difficult to study and modify in a purely theoretical or real-world hardware context.

OS161 was specifically designed at Harvard University to circumvent this complexity. It is an instructional, Unix BSD-like operating system written in C that prioritizes code readability and pedagogical value over raw performance.

It provides a foundational layer including low-level trap and interrupt handling, device drivers, in-kernel threads, a baseline scheduler, a minimal virtual memory system, and a rudimentary file system. By working with OS161, we gain a hands-on, realistic environment to practice operating system development without being bogged down by legacy hardware quirks.

1.1.1 What is an Emulator?

To ensure a consistent, safe, and easily debuggable hardware experience, OS161 does not run on "bare metal" (the physical hardware of your laptop). Instead, it is executed atop a virtual machine known as **sys161**.

An emulator is a specialized software application that perfectly mimics the internal architecture of a specific physical CPU and its peripheral devices. Sys161 specifically simulates a **MIPS R3000 microprocessor**. The MIPS architecture was chosen because its Reduced Instruction Set Computer (RISC) design is clean, elegant, and highly educational, completely avoiding the historical and structural complexities associated with architectures like x86.

It is crucial to distinguish between the two distinct entities:

- **sys161** is the **emulator software**. It runs natively on your host machine (e.g., Linux or macOS). It provides the fake CPU, fake RAM, and fake hard drives.
- **os161** is the **actual operating system code**. When compiled, it becomes a binary executable (e.g., `kernel-DUMBVM`).

When the command `sys161 kernel` is executed, the emulator boots up the simulated MIPS machine and loads your compiled OS executable into its simulated RAM, just like a real motherboard loading an OS from a hard drive.

1.1.2 Cross-Compilation Explained

Because your host machine (likely an x86_64 or ARM Mac/PC) has a fundamentally different architecture than the target machine (MIPS), you cannot use your system's default `gcc` compiler to build the OS161 kernel. If you did, it would generate x86/ARM machine code, which the MIPS emulator cannot understand.

This necessitates **cross-compilation**. Cross-compilation is the process of building executable code for a platform other than the one on which the compiler is running. The OS161 toolchain includes a specially built version of GCC (`mips-harvard-os161-gcc`). When you compile your C code using this specific compiler, it runs on your host machine but translates the C code directly into MIPS assembly and MIPS machine code.

1.2 Project Structure Overview

Understanding where files are located is paramount before modifying the kernel. Imagine the OS161 project as being split into three major domains: the cross-compilation tools, the source code, and the deployment root.

```
os161/
|-- tools/                # The cross-compilation toolchain (MIPS GCC, GDB, bmake, sys161)
|-- os161-base-2.0.2/     # The actual source code of the operating system
|   |-- kern/             # 1. KERNEL SOURCE CODE (Where you will work)
|   |   |-- arch/mips/    # Low-level MIPS architecture code (traps, assembly)
|   |   |-- compile/      # Build directories generated by ./config
|   |   |-- conf/         # Configuration files (conf.kern, DUMBVM)
|   |   |-- include/      # Kernel header files (.h)
|   |   |-- main/         # Kernel entry point (main.c)
|   |   |-- proc/         # Process management code (proc.c)
|   |   |-- syscall/      # System call implementations (you will create file_syscalls.c)
|   |   |-- thread/       # Thread management and synchronization (synch.c)
|   |   |-- vm/           # Virtual memory implementation (dumbvm.c)
|   |-- userland/         # 2. USER-SPACE PROGRAMS (C standard library, test binaries)
|-- root/                 # 3. THE SIMULATED HARD DRIVE
                          # This is where the compiled kernel and user binaries are
                          # installed. Sys161 boots from this folder.
```

1.3 The OS161 Build System

Building the OS161 kernel involves a strict, sequential four-step pipeline managed by BSD make (`bmake`).

1.3.1 1. Configuring the Kernel (./config)

Before any compilation can occur, the build tree must be generated. This is achieved by navigating to the `kern/conf` directory and executing `./config DUMBVM` (or your specific configuration name). The `config` script reads the master `conf.kern` file (which defines all possible options and files) and the specific `DUMBVM` file (which selects which options to activate). It then generates a dedicated compile directory (`kern/compile/DUMBVM`) populated with the necessary Makefiles and auto-generated headers.

1.3.2 2. Building the Dependency Graph (**bmake depend**)

Once configured, we navigate to the generated compile directory (**kern/compile/DUMBVM**) and execute **bmake depend**. This crucial step invokes the C preprocessor to scan all source files for **#include** directives. It builds a comprehensive dependency graph (mapping which **.c** files rely on which **.h** files). This ensures that during subsequent rebuilds, if you modify a single header file, **bmake** knows exactly which source files need to be recompiled, saving immense amounts of time.

1.3.3 3. Compilation and Linking (**bmake**)

Running **bmake** performs the actual cross-compilation. It translates the C source code into MIPS object files (**.o**) and links them together to generate the final binary executable named **kernel-DUMBVM**.

1.3.4 4. Installation (**bmake install**)

Compiling alone is insufficient. The **kernel-DUMBVM** executable currently sits in the compile directory. The emulator, however, is configured to boot from the **root** directory. The command **bmake install** copies the newly generated executable into the **root** directory and updates a symbolic link named **kernel** to point to it, ensuring the emulator boots the fresh kernel rather than a stale version.

1.3.5 VSCode Build and Debug Integration

While the traditional terminal workflow is fundamental to understanding the build system, modern development often utilizes IDEs. A correctly configured Visual Studio Code environment maps these terminal commands to VSCode Tasks:

- **Terminal → Run Task → Run Config:** Maps to **./config**. You select the configuration (e.g., **HELLO**, **DUMBVM**) and it generates the **kern/compile/** directory.
- **Terminal → Run Task → Make depend:** Maps to **bmake depend**.
- **Terminal → Run Task → Build and Install:** Maps to **bmake** followed by **bmake install**.

For debugging directly in VSCode:

1. Execute **Terminal → Run Task → RUN OS161**. This starts the simulator in listening mode (**sys161 -w kernel**).
2. Execute **Run → Start Debugging**. This attaches the GDB debugger.
3. You can now set breakpoints visually (e.g., inside **kmain**) and step through the code.

1.4 Adding a Custom Boot Message (Configuration Deep Dive)

A core exercise in this laboratory demonstrates the modular nature of the OS161 build system by adding a custom "Hello" message during the boot sequence. This introduces the concept of **Kernel Configuration Options**.

When you modify the master configuration file **kern/conf/conf.kern**, you define new features:

```
1 // In kern/conf/conf.kern
2 defoption hello
3 optfile hello main/hello.c
```

How it works:

- `defoption hello`: This tells the build system that an option named "hello" exists. During the `./config` step, the script will *always* auto-generate a header file named `opt-hello.h`. If the option is enabled in the specific configuration file (e.g., `DUMBVM`), the header will contain `#define OPT_HELLO 1`. If it is disabled or absent, the header will contain `#define OPT_HELLO 0`.
- `optfile hello main/hello.c`: This tells the build system that the source file `main/hello.c` should *only* be compiled and linked into the final kernel if the "hello" option is enabled.

We then provide the implementation by creating `hello.h` and `hello.c`.

```

1 // In kern/main/hello.c
2 #include <types.h>
3 #include <lib.h>
4 #include "hello.h"
5
6 void hello(void) {
7     kprintf("Hello from the new OS161 branch!\n");
8 }

```

To execute this function during boot, we integrate it into the main kernel initialization routine located in `kern/main/main.c`. It is critical to wrap both the function call and its include directive in a preprocessor conditional checking the auto-generated macro.

```

1 // In kern/main/main.c
2 #include "opt-hello.h" // ALWAYS included
3
4 #if OPT_HELLO
5 #include "hello.h"      // Included ONLY if OPT_HELLO is 1
6 #endif
7
8 void kmain(char *arguments) {
9     boot();
10
11 #if OPT_HELLO
12     hello();             // Executed ONLY if OPT_HELLO is 1
13 #endif
14
15     menu(arguments);
16 }

```

By wrapping the call in `#if OPT_HELLO`, we ensure that if the configuration is disabled, the compiler ignores the block. If we didn't use this `#if` check, disabling the option would prevent `hello.c` from being compiled, leading to a fatal Linker Error because `kmain` would be trying to call an undefined `hello()` function.

The #ifdef vs #if Trap

A very common pitfall (and a frequent exam question) is attempting to use `#ifdef OPT_HELLO` instead of `#if OPT_HELLO`.

In C, `#ifdef` checks whether a macro is *defined at all*, regardless of its actual value. As we established earlier, the `./config` script **always** generates the `opt-hello.h` file, and it **always** defines the macro.

- If enabled: `#define OPT_HELLO 1`
- If disabled: `#define OPT_HELLO 0`

Because the macro is always defined, `#ifdef OPT_HELLO` will **always evaluate to TRUE**. If the option was disabled in your configuration, the compiler will attempt to compile the `hello()` function call anyway. But since `hello.c` wasn't compiled, the linker will throw a fatal undefined reference error.

Therefore, you must strictly use `#if OPT_HELLO`, which checks the *mathematical value* of the macro (1 or 0).

1.5 Distributed Kernel Debugging

Debugging an operating system kernel presents unique challenges. You cannot simply run a debugger *inside* an OS that is currently crashing. OS161 utilizes a **distributed debugging model**.

- **The Target:** The emulator is started with a flag instructing it to pause and wait for a debugger connection: `sys161 -w kernel`. This opens a local UNIX domain socket and freezes the simulated CPU before the very first instruction is executed.
- **The Host:** In a separate terminal, the cross-compiled debugger is launched: `mips-harvard-os161-gdb kernel`.
- **The Connection:** Within GDB, the command `target remote unix:.sockets/gdb` establishes the connection to the paused emulator, allowing for breakpoints, stepping, and state inspection over the socket.

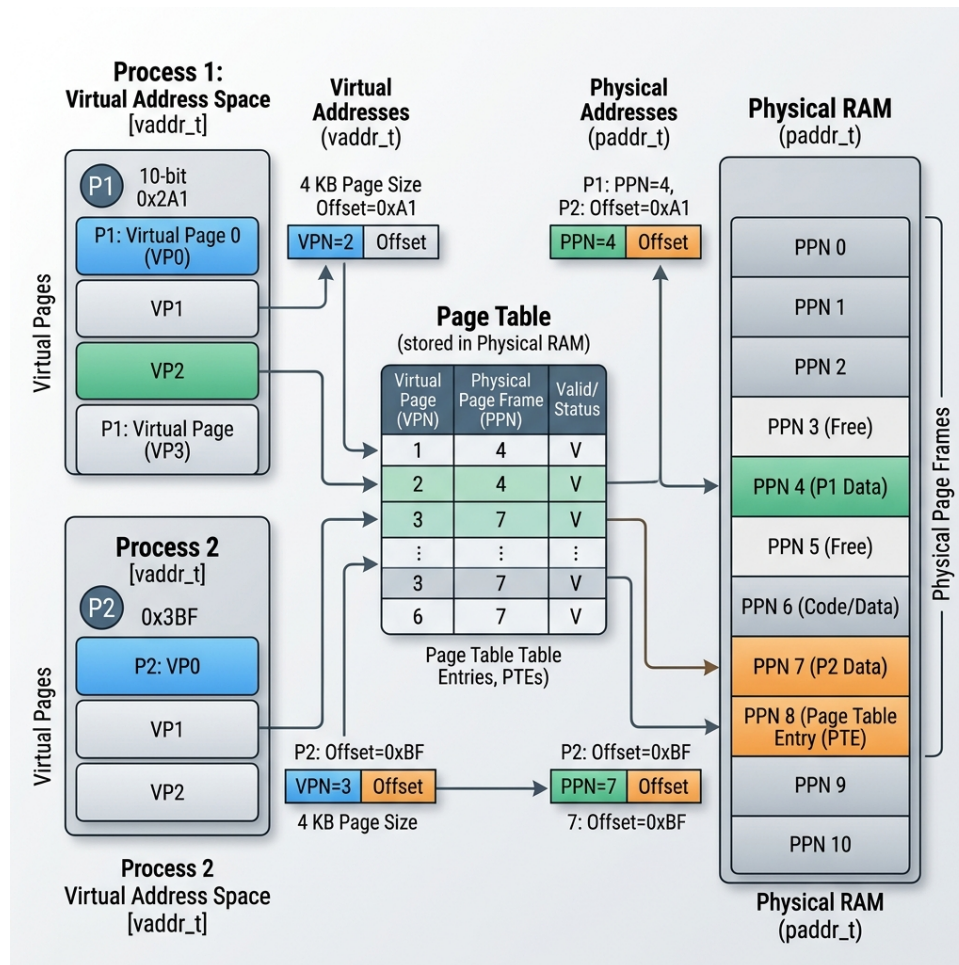
When debugging concurrent threads (e.g., during `thread_fork`), special care must be taken. Stepping *over* a fork instruction keeps the debugger attached to the parent thread, while the newly spawned thread executes freely in the background. To debug the child thread, a breakpoint must be explicitly set on its entry function before allowing the parent to continue execution.

1.6 Core OS161 Data Types: `vaddr_t` vs `paddr_t`

When working with kernel memory, it is vital to distinguish between virtual and physical addresses.

- **`paddr_t` (Physical Address):** Represents an actual, hardware-level address in the physical RAM. It is strictly managed by the kernel and cannot be accessed directly by user processes. It is used when interfacing with low-level hardware structures or during manual memory mapping.
- **`vaddr_t` (Virtual Address):** Represents the address as seen by a process or by the kernel itself.

Every process operates within its own isolated virtual address space. When a process attempts to access a `vaddr_t`, the Memory Management Unit (MMU) translates it to a corresponding `paddr_t` using a Page Table. This abstraction means that the exact same `vaddr_t` in Process A and Process B will translate to entirely different `paddr_t` physical locations.

Figure 1.1: Mapping Virtual Addresses (`vaddr_t`) to Physical Addresses (`paddr_t`).

1.7 Introduction to OS161 Concurrency

1.7.1 Processes vs Threads

A **program** is a passive entity on disk. When it is executed, it becomes a **process**—an active entity containing executable code, a data section, a heap, an execution stack, and a Program Counter (PC).

A **thread** is a single flow of execution within a process. A process can contain multiple threads that share the same code, global data, and open files, but each thread maintains its own:

- Program Counter (PC)
- CPU Register state
- Execution Stack

OS161 primarily manages **kernel threads**. The kernel maintains an array or list of thread structures. When a thread is suspended, its execution context (registers and PC) is saved into a **switchframe** on its stack.

1.7.2 The Thread API

The OS161 thread management library relies on a **struct thread** containing the thread's name, synchronization primitives, state, stack pointer, and a pointer to its parent process's Process

Control Block (PCB).

Key functions include:

- `thread_fork()`: Creates a new thread. It allocates a new `struct thread`, initializes the `switchframe` with `switchframe_init()`, and places the thread into the ready queue using `thread_make_runnable()`.
- `thread_yield()`: Voluntarily suspends the currently running thread, yielding the CPU. It places the thread back in the ready queue (`S_READY`) and calls `thread_switch()` to execute the next thread.
- `thread_exit()`: Terminates the current thread.
- `thread_switch()`: Performs the low-level context switch by swapping the `switchframe` of the current thread with the next one.

1.7.3 Testing and Debugging Threads

OS161 provides built-in commands to test thread concurrency from the kernel menu:

- `tt1`: Creates `NTHREADS` (usually 8) and runs them in "quiet" mode, where each thread prints a single identifying character.
- `tt2`: Similar to `tt1`, but runs in "loud" mode, printing the character 120 times.
- `tt3`: A complex test involving thread synchronization. It uses two sets of threads—one that sleeps for random intervals, and another that performs matrix math.

Because threads are scheduled concurrently by the kernel (via `hardclock` interrupts calling `thread_yield`), their execution order is non-deterministic. Threads 0 through 7 will not execute in sequential order.

When debugging threads using GDB, setting breakpoints on `thread_create`, `thread_fork`, `thread_yield`, and `thread_switch` allows you to trace the lifecycle and context switching of these concurrent execution flows.

1.8 Laboratory 01: Print hello world in OS161

1.8.1 High-Level Overview

The primary objective of Laboratory 01 is to familiarize yourself with the OS161 build system and the kernel configuration process. Instead of modifying core OS mechanics, you will implement a custom modular feature: a greeting message that prints during the kernel boot sequence.

To achieve this safely without breaking the build, you must follow the strict OS161 pipeline:

1. Define a new kernel option (`hello`) in the master configuration file.
2. Enable the option for your specific kernel build.
3. Create the actual C source code and header file for the greeting.
4. Integrate the function call safely into the kernel's main boot routine.
5. Reconfigure the build system to generate new headers and Makefiles.
6. Cross-compile and deploy the modified kernel to the simulated MIPS hardware.

1.8.2 Step 1: Define the Option in `conf.kern`

First, we must tell the OS161 build system that a new optional feature exists and associate it with a source file.

- **Action:** Open the master configuration file at `kern/conf/conf.kern`.
- **Addition:** Scroll to the very bottom and add the following lines:

```
1 defoption hello
2 optfile hello main/hello.c
```

- **Why it is needed:** `defoption hello` ensures the `./config` script generates the `opt-hello.h` macro file. `optfile` tells the compiler that the file `hello.c` belongs to this option and should only be compiled if the option is activated.

1.8.3 Step 2: Enable the Option in DUMBVM

Defining the option doesn't activate it. We must explicitly turn it on for our specific kernel build.

- **Action:** Open your target configuration file at `kern/conf/DUMBVM`.
- **Addition:** Add the following line at the bottom:

```
1 options hello
```

- **Why it is needed:** This line instructs the configuration script to set the auto-generated macro `OPT_HELLO` to 1 instead of 0, effectively enabling the feature.

1.8.4 Step 3: Create the Source Code and Header File

Now we write the actual logic for the greeting.

- **Action:** Create a new header file `kern/include/hello.h` and define the function blueprint:

```
1 #ifndef _HELLO_H_
2 #define _HELLO_H_
3
4 void hello(void);
5
6 #endif /* _HELLO_H_ */
```

- **Action:** Create a new source file `kern/main/hello.c` and implement the function:

```
1 #include <types.h>
2 #include <lib.h>
3 #include "hello.h"
4
5 void hello(void) {
6     kprintf("Hello from the new OS161 branch!\n");
7 }
```

- **Why it is needed:** Separating prototypes (`.h`) from implementations (`.c`) is a strict requirement in C and OS161 to satisfy the compiler's `-Wmissing-prototypes` safety flag.

1.8.5 Step 4: Integrate into the Boot Sequence

The kernel must actually execute your function when it boots up.

- **Action:** Open the main kernel file at `kern/main/main.c`.
- **Addition (Top):** Add the includes near the top of the file, alongside the other includes:

```
1 #include "opt-hello.h"
2
3 #if OPT_HELLO
4 #include "hello.h"
5 #endif
```

- **Addition (Inside `kmain`):** Locate the `kmain(char *arguments)` function. Add your call right after `boot()`; and before `menu(arguments);`:

```
1 #if OPT_HELLO
2     hello();
3 #endif
```

- **Why it is needed:** The `#if OPT_HELLO` wrapper ensures your code is safely ignored by the compiler if you ever decide to disable the `hello` option. Without it, disabling the option would cause a fatal linker error.

1.8.6 Step 5: Reconfigure the Build System

Because we added new configuration flags and created new files, the internal Makefiles must be regenerated.

- **Action:** Open your terminal, navigate to the configuration directory, and run the script:

```
cd ~/os161/os161-base-2.0.3/kern/conf
./config DUMBVM
```

- **Why it is needed:** This parses your `DUMBVM` file and generates the crucial `opt-hello.h` header file, injecting it into your compile directory.

1.8.7 Step 6: Compile and Install

Now we translate the C code into MIPS machine code.

- **Action:** Navigate to the compilation directory and run the build sequence:

```
cd ../compile/DUMBVM
bmake depend
bmake
bmake install
```

- **Why it is needed:**

- `bmake depend` maps the relationships of your new `hello.h` file.
- `bmake` performs the actual cross-compilation into a MIPS executable.
- `bmake install` moves the resulting `kernel` binary into the `/os161/root` directory so the emulator can find it.

1.8.8 Step 7: Boot the Simulator

Finally, verify that your implementation was successful.

- **Action:** Navigate to the root directory and start the OS:

```
cd ~/os161/root
sys161 kernel
```

- **Why it is needed:** The simulator will execute the kernel, and you should now see your custom greeting printed directly to the terminal right before the OS161 menu prompt appears!

Configuration and Build Flowchart

To summarize the entire process of configuring and compiling the OS161 kernel, the following flowchart illustrates the workflow. It shows what happens when `./config` is executed, how the configuration options are translated into C macros, and the subsequent compilation steps.

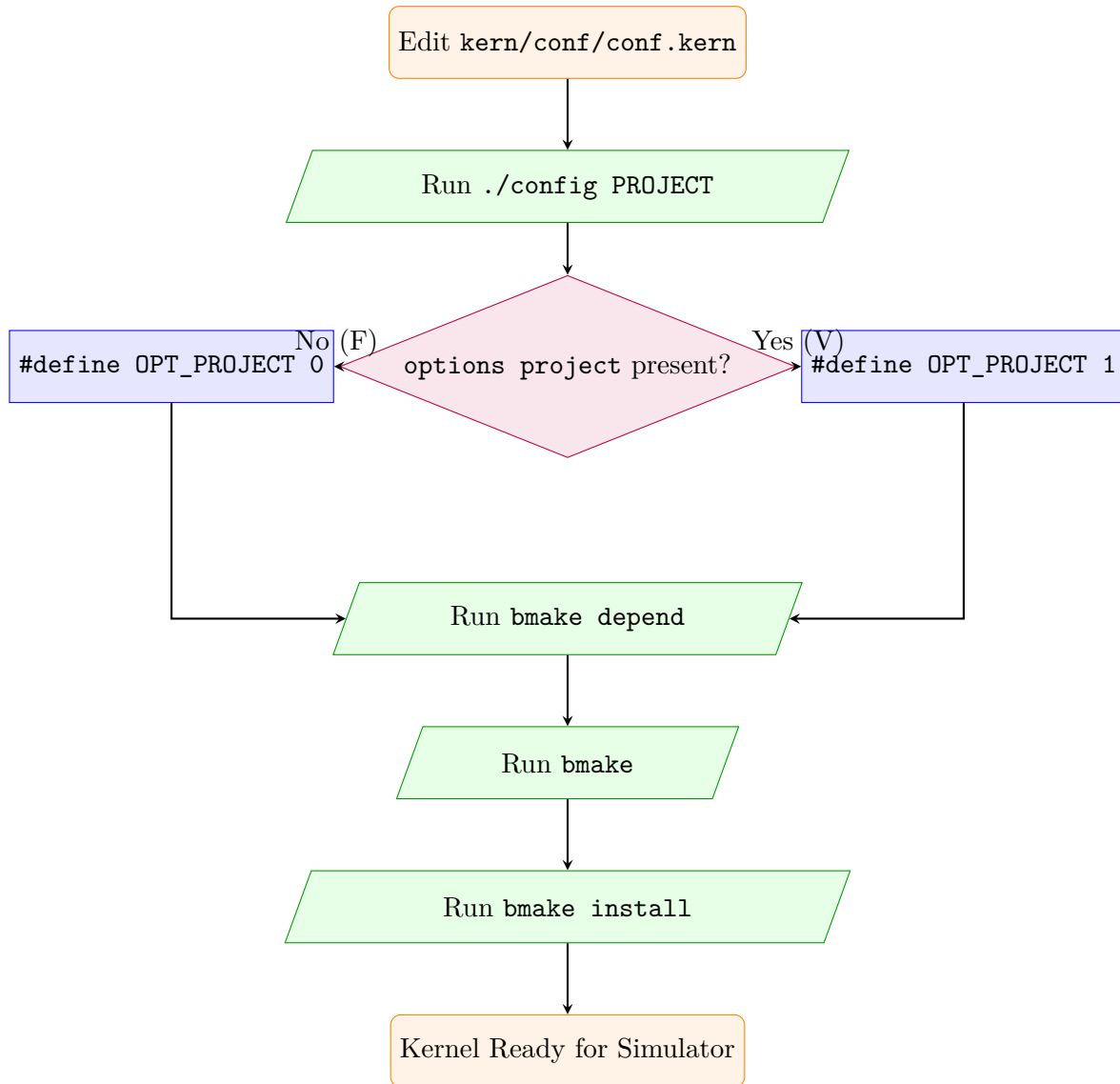


Figure 1.2: OS161 Kernel Configuration and Build Workflow

Past Exam Questions

[Exam General - Q1] Describe the significance of the `bmake depend` step in the compilation pipeline. Why is it necessary, and what happens if it is omitted when a header file is modified?

Answer: The `bmake depend` command invokes the compiler's preprocessor to scan all C source files for `#include` directives. Its purpose is to construct a comprehensive dependency graph mapping which `.c` files rely on which `.h` header files.

It is necessary for optimizing the build process. When `bmake` is subsequently run, it uses this graph to recompile *only* the `.c` files that depend on recently modified headers, rather than

recompiling the entire millions of lines of OS code.

If `bmake depend` is omitted after modifying a header file (e.g., changing a struct definition or a function prototype), the build system remains unaware of the structural change. Consequently, `bmake` will skip recompiling the source files that include that header. This results in a mismatched kernel binary where different modules are compiled with conflicting definitions of memory structures, inevitably causing catastrophic memory corruption, undefined behavior, or kernel panics during runtime.

[Exam 2025/01/13 - Q6] Configurazione File in OS161 (Configurazione del Kernel e Compile)

È dato un Sistema OS161. Si supponga di aver aggiunto le istruzioni seguenti a `kern/conf/conf.kern`:

```
defoption project
optfile project syscall/project.c
```

e di aver creato il file `PROJECT` in `kern/conf`, copiato dal file `DUMBVM`.

A) Si dica se le azioni descritte, più l'esecuzione (in `kern/conf`) di `./config PROJECT`, sono sufficienti affinché il file opzionale `syscall/project.c` sia compilato quando si eseguono (in `kern/compile/PROJECT`) i comandi `bmake depend` e `bmake`.

Answer: No, non sono sufficienti. Le istruzioni in `conf.kern` si limitano a *definire* l'esistenza di una nuova opzione chiamata `project` e ad associarvi il file sorgente `project.c`. Tuttavia, per far sì che il file venga effettivamente compilato, l'opzione deve essere esplicitamente *attivata* nel file di configurazione specifico del kernel. È strettamente necessario aggiungere la riga `options project` all'interno del file `kern/conf/PROJECT` *prima* di eseguire il comando `./config PROJECT`. Senza questa riga, l'opzione è considerata disabilitata e il build system ignorerà il file sorgente.

B) Quale file, tra `project.h` e `opt-project.h` viene generato automaticamente dal comando `./config PROJECT`? Il file viene sempre generato, oppure solo se l'istruzione `options project` compare nel file `PROJECT`?

Answer: Il comando `./config PROJECT` genera automaticamente *solo* il file header `opt-project.h`. Questo file viene *sempre* generato, in ogni caso, indipendentemente dal fatto che l'istruzione `options project` sia presente (attivata) o assente (disattivata) nel file `PROJECT`. Il file `project.h` (che conterrebbe le firme delle funzioni come `project_init`) NON viene mai generato automaticamente; deve essere scritto manualmente dallo sviluppatore e posizionato nella cartella `kern/include/`.

C) Cosa contiene il file (quello generato automaticamente, citato nella domanda precedente)?

Answer: Il file `opt-project.h` contiene esclusivamente una macro del pre-processore in linguaggio C.

- Se l'opzione è stata attivata inserendo `options project` nel file `PROJECT`, conterrà: `#define OPT_PROJECT 1`
- Se l'opzione non è stata attivata (o è commentata), conterrà: `#define OPT_PROJECT 0`

D) Si supponga di inserire in `main.c` l'istruzione `project_init()`; . Considerando che la funzione `project_init()` è implementata nel file `syscall/project.c`, come si può fare in modo che l'istruzione sia considerata e compilata solo nelle versioni del kernel in cui l'opzione `project` è abilitata?

Answer: Per evitare errori in fase di linking (funzione non definita) quando l'opzione è disabilitata, è necessario circondare sia l'inclusione dell'header file sia la chiamata alla funzione con delle direttive condizionali del pre-processore (`#if`), controllando il valore della macro definita nel file auto-generato `opt-project.h`.

Nel file `kern/main/main.c` bisogna scrivere:

```

1 #include "opt-project.h" // Includi sempre questo file
2
3 #if OPT_PROJECT
4 #include "project.h" // Includi l'header manuale solo se l'opzione e' attiva
5 #endif
6
7 // ... all'interno della funzione kmain ...
8 #if OPT_PROJECT
9     project_init(); // Esegui la funzione solo se l'opzione e' attiva
10 #endif

```

[Exam 2024-07-11 - Q16] Configurazione conf.kern (English version)

An OS161 system is given. Suppose you have added the following instructions to `kern/conf/conf.kern`:

```

defoption project
optfile project syscall/project.c

```

and you have created the `PROJECT` file in `kern/conf`, copied from the `DUMBVM` file.

A) Tell if the actions described above, plus the execution (in `kern/conf`) of `./config PROJECT`, are sufficient for the optional file `syscall/project.c` to be compiled when you execute (in `kern/compile/PROJECT`) the commands `bmake depend` and `bmake`.

Answer: No, they are not sufficient. While you have defined the option globally, you must explicitly enable it for this specific kernel build. You must add the line `options project` inside the `kern/conf/PROJECT` file *before* executing `./config PROJECT`. Without this line, the build system considers the option disabled and will completely ignore `syscall/project.c`.

B) Which one between the `project.h` and `opt-project.h` files is automatically generated by the command `./config PROJECT`? Is it always generated, or only if the following instruction appears in `PROJECT`? (`options project`)

Answer: The command `./config PROJECT` automatically generates *only* the `opt-project.h` file. It is **always generated**, unconditionally, regardless of whether `options project` is present in the configuration file or not. The `project.h` file is never generated automatically; developers must create it manually.

C) What does the file contain (the automatically generated one, referred by the previous question)?

Answer: The automatically generated file (`opt-project.h`) contains a single C preprocessor macro. If the option was enabled (i.e., `options project` was in the config file), it contains `#define OPT_PROJECT 1`. If it was absent or disabled, it contains `#define OPT_PROJECT 0`.

[Exam 2021-02-15 & 2021-10-18 - Domanda 4] Configurazione e Compilazione Kernel (The #ifdef Trap)

Sia dato un sistema OS161. Si supponga di aver aggiunto a `kern/conf/conf.kern` le istruzioni:

```

defoption exam
optfile exam syscall/exam.c

```

e di aver creato in `kern/conf` il file `EXAM`, copiato dal file `DUMBVM`. (Le domande A, B e C sono omesse qui poiché identiche alle precedenti, con la variabile `exam` al posto di `project`.)

D) Si supponga di inserire, nel file `main.c`, l'istruzione `exam_init()`; . Tenendo conto che è realizzata nel file `syscall/exam.c`, come è possibile fare in modo che l'istruzione sia presa in considerazione e compilata solamente nelle versioni del kernel in cui viene abilitata l'opzione `exam`, mediante le istruzioni seguenti?

```

1 #ifdef OPT_EXAM
2     exam_init();
3 #endif

```

Dire anche cosa è necessario includere in un file C per poter usare questo tipo di opzioni.

Answer: Le istruzioni fornite nel testo della domanda **contengono un grave errore logico**. Utilizzare `#ifdef OPT_EXAM` è scorretto perché il file auto-generato definisce *sempre* la macro `OPT_EXAM` (assegnandole valore 1 o 0). Poiché la macro è sempre definita, la direttiva `#ifdef` valuterà sempre a VERO, causando un errore in fase di linking se l'opzione è disabilitata (poiché `exam.c` non verrebbe compilato).

Per risolvere il problema, le direttive devono essere modificate utilizzando `#if`, che valuta il valore matematico della macro:

```
1 #if OPT_EXAM
2     exam_init();
3 #endif
```

Inoltre, per poter utilizzare questo tipo di opzioni, è **strettamente necessario** includere in cima al file C il file header generato automaticamente dal sistema di configurazione. Nel file `main.c` bisogna aggiungere la riga:

```
1 #include "opt-exam.h"
```

[Exam 2020-06-27 - Domanda 4] Configurazione Kernel

Nota: Questa domanda è strutturalmente identica all'esame 2025-01-13 e 2024-07-11, ma utilizza la dicitura `exam` invece di `project`. L'unica differenza è la domanda D che non propone l'uso errato di `#ifdef`, ma chiede genericamente come fare, risolvibile con il classico `#if OPT_EXAM` e `#include "opt-exam.h"`.

Chapter 2

LAB 02: Running User Programs and System Calls

2.1 The Need for System Calls

The primary objective of an operating system is to safely and efficiently execute user-level programs. In the base distribution of OS161, user programs cannot execute correctly because the boundary separating user-space from kernel-space is incomplete.

Specifically, when a user program attempts to perform input/output operations or attempts to terminate gracefully, the system crashes. For example, executing the palindrome test program from the OS161 menu (`p testbin/palin`) results in a kernel panic indicating "Unknown syscall 55" followed by "Unknown syscall 3".

These failures occur because the MIPS emulator traps into the kernel upon encountering a `syscall` instruction, but the kernel lacks the implementation to handle these specific requests. Syscall 55 corresponds to `SYS_write`, which is invoked by the C standard library's `printf` function to output text. Syscall 3 corresponds to `SYS__exit`, which is implicitly called at the termination of the `main()` function to instruct the operating system to clean up the process's resources. Without these, user programs cannot communicate with the console or terminate without bringing down the entire system.

2.2 The Trap Mechanism: Trapframe vs Switchframe

To understand how system calls are intercepted, one must understand how the kernel handles execution state. In OS161, there are two primary structures used to save the CPU registers, but they serve completely different purposes and are triggered by different events: the **Trapframe** and the **Switchframe**.

2.2.1 The Trapframe

! EXAM FOCUS: IMPLEMENTATION REQUIRED

Function: `thread_fork(const char *name, struct proc *proc, ...)`

Exam Task: *Line-by-Line Analysis & Trapframe*

You must explain what the `entrypoint` is, clarify that the allocated `STACK_SIZE` is for the kernel stack (not user stack), and determine whether `thread_make_runnable` executes the thread immediately (No, it just queues it). Also, be able to distinguish whether a `trapframe` or `switchframe` is used.

A **trapframe** is automatically created when the hardware transitions from user mode to kernel mode due to an exception (such as a system call, an interrupt, or a fault like accessing invalid memory). When a user program executes the MIPS `syscall` instruction, the hardware forces a trap. The low-level assembly handler (`mips_trap`) immediately pushes a trapframe onto the **kernel stack** of the currently running thread. This trapframe contains the exact state of all user-space registers at the exact moment the trap occurred. This is vital because the kernel will now use the CPU, overwriting registers; the trapframe ensures that when the system call finishes, the kernel can perfectly restore the user program's state so it can resume execution seamlessly.

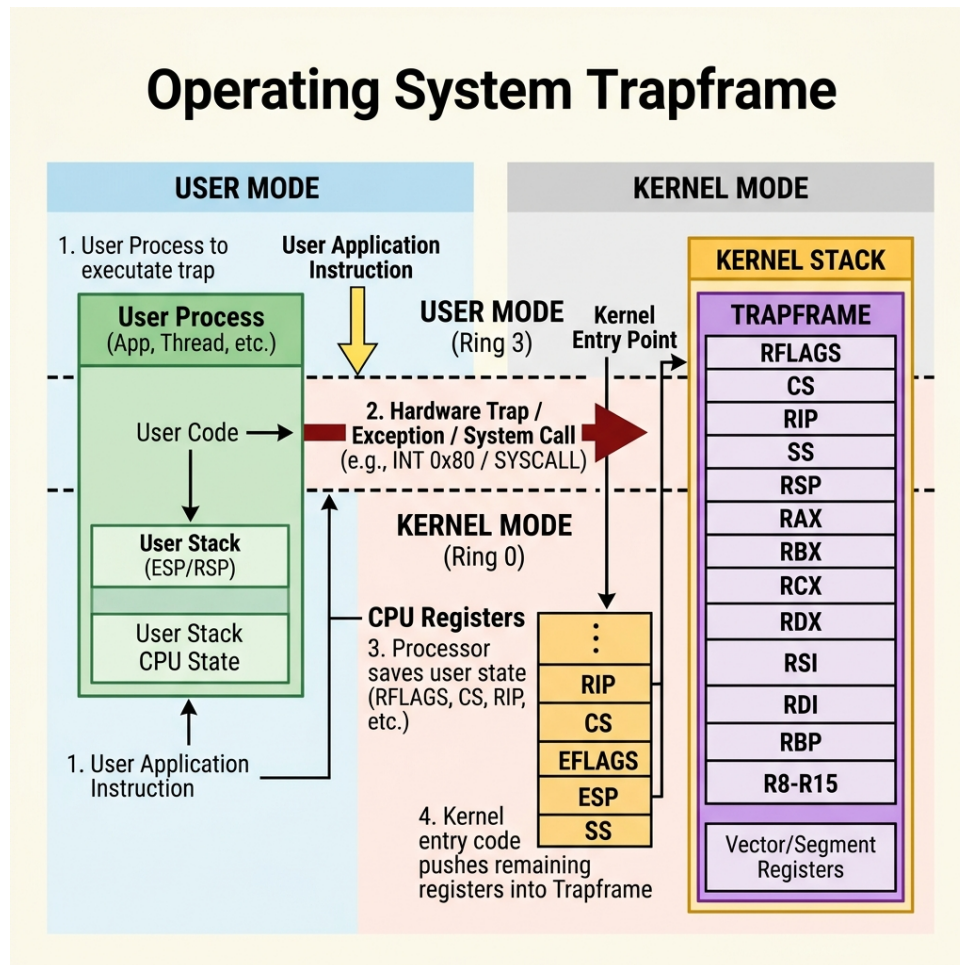


Figure 2.1: Exception Handling and Trapframe Creation

2.2.2 The Switchframe

A **switchframe**, on the other hand, is a purely software-driven construct. It is used exclusively during a **context switch** between two kernel threads (i.e., when `thread_switch` is called). Unlike a trapframe which saves user state, a switchframe saves the *kernel* execution state (specifically, the callee-saved registers required by the C calling convention). When a thread voluntarily yields the CPU (or is preempted), it pushes a switchframe onto its **kernel stack**. When the scheduler later selects this thread to run again, it pops the switchframe off the stack, restoring the kernel registers and resuming execution exactly where it left off inside the kernel.

In summary, a trapframe handles the user-to-kernel boundary (hardware-driven), while a switchframe handles the kernel-to-kernel thread switching boundary (software-driven). Both are allocated on the **kernel stack** of the respective thread.

2.3 Process Execution: From Menu to User Mode

Understanding how a user program is actually launched requires tracing the execution path from the kernel menu. When the command `p testbin/palin` is issued, the `cmd_prog` function is invoked. This function acts as a wrapper, calling `common_prog`, which sets up the new process environment.

The execution trace follows this specific flow:

```
[Kernel Menu Thread]
|___ cmd_prog("p testbin/palin")
|___ common_prog()
|
|--- 1. proc_create_runprogram()
|      (Allocates raw process data structure)
|
|--- 2. thread_fork("cmd_progthread")
|      (Spawns a new kernel thread attached to the process)
|
[New Kernel Thread Execution]
|___ cmd_progthread()
|___ runprogram("testbin/palin")
|
|--- 3. vfs_open()
|      (Opens the executable file from disk)
|
|--- 4. as_create() & load_elf()
|      (Parses ELF headers, allocates physical memory frames,
|       and loads code/data segments into RAM)
|
|--- 5. as_define_stack()
|      (Allocates the user stack area)
|
|___ 6. enter_new_process()
|      (Forges a Trapframe, drops privileges, and executes
|       an assembly jump into User Mode!)
```

2.3.1 Deep Dive: `load_elf` and Executable Loading

When `runprogram` launches a user program, it opens the binary file via `vfs_open`. The binary is formatted as an **ELF (Executable and Linkable Format)** file. The `load_elf` function is responsible for parsing this file, configuring the virtual address space, and loading its contents into physical RAM.

The ELF file is composed of various segments. The primary ones include:

- **Code Segment:** Contains `.text` (the actual executable instructions) and `.rodata` (read-only global data).
- **Data Segment:** Contains `.data` (initialized global data), `.bss` (uninitialized global data), and `.sbss` (small uninitialized global data).

Understanding the internal structure of `load_elf` is critical for exam preparation, as it orchestrates the memory setup. Here is the high-level implementation structure:

```

1 int load_elf(struct vnode *v, vaddr_t *entrypoint) {
2     Elf_Ehdr eh;    // ELF Header
3     Elf_Phdr ph;    // Program Header
4     struct addrspace *as = proc_getas();
5
6     /* 1. Read and verify ELF Header magic number */
7     read_elf_header(v, &eh);
8
9     /* 2. Define Logical Regions */
10    for (int i = 0; i < eh.e_phnum; i++) {
11        read_program_header(v, &ph);
12        if (ph.p_type == PT_LOAD) {
13            /* Sets logical addresses and sizes for Code/Data segments */
14            as_define_region(as, ph.p_vaddr, ph.p_memsz, ...);
15        }
16    }
17
18    /* 3. Allocate Physical Memory */
19    as_prepare_load(as);
20
21    /* 4. Load Segments from Disk to RAM */
22    for (int i = 0; i < eh.e_phnum; i++) {
23        read_program_header(v, &ph);
24        if (ph.p_type == PT_LOAD) {
25            /* Reads the actual Code or Data segment from the ELF file */
26            load_segment(as, v, ph.p_offset, ph.p_vaddr, ...);
27        }
28    }
29
30    /* 5. Finalize the Address Space */
31    as_complete_load(as);
32
33    *entrypoint = eh.e_entry;
34    return 0;
35 }

```

The Core Memory Loading Functions

As seen in the skeleton above, `load_elf` delegates its heavy lifting to four critical functions. Understanding their exact responsibilities and execution order gives logical sense to the process creation chain (and is a frequent exam topic, e.g., Exam 2024-07-11 Q4.B):

1. **as_define_region**: Called in the initial loop for each Program Header, this function is responsible for **setting the logical addresses and dimensions** of the code and data segments. It registers the starting virtual address (`VStart`) and the region's size inside the `addrspace` structure, but it does *not* allocate physical RAM yet.
2. **as_prepare_load**: After all logical regions are defined, this function is called to **allocate the address space in physical memory**. It officially secures the physical RAM frames necessary to host the process. In `dumbvm`, this is where `getppages()` is triggered under the hood.
3. **load_segment (For Code)**: This function is responsible for the **physical transfer of bytes** from the ELF file on disk (via the `vnode`) directly into the newly allocated RAM.
4. **load_segment (For Data)**: *Attention*: `load_segment` is a highly generic function. The ELF loader makes absolutely no distinction between code and data segments at load time. It utilizes the exact same `load_segment` function to transfer *any* loadable segment from the disk into memory.

5. **as_complete_load**: This function is called at the very end of the entire loading process. Its main purpose is to finalize the setup, which usually involves configuring memory permissions (such as enforcing read-only protections on the newly loaded code segment).

Finally, `load_elf` returns the `entrypoint` address, which `enter_new_process` will push into the `epc` register of the Trapframe so the CPU knows where to start executing in User Mode.

2.4 The Syscall Dispatcher

When the user code eventually needs kernel assistance—such as printing a character—it executes the MIPS `syscall` instruction. This hardware trap pushes the trapframe and routes execution to the `syscall(struct trapframe *tf)` dispatcher function located in `kern/arch/mips/syscall/syscall.c`.

In the MIPS calling convention, the first four arguments to a function or system call are stored in registers `a0`, `a1`, `a2`, `a3`, and the system call identifier (the `callno`) itself is stored in register `v0`.

The dispatcher uses a `switch` statement based on `tf->tf_v0` to route the call to the appropriate handler (e.g., `sys_write`).

```

1 // Modifications inside syscall(struct trapframe *tf) in kern/arch/mips/syscall
  /syscall.c
2 #if OPT_SYSCALLS
3     case SYS_write:
4         retval = sys_write((int)tf->tf_a0,
5                             (userptr_t)tf->tf_a1,
6                             (size_t)tf->tf_a2);
7         if (retval < 0) err = ENOSYS;
8         else err = 0;
9         break;
10
11     case SYS__exit:
12         sys__exit((int)tf->tf_a0);
13         /* sys__exit does not return; the thread is destroyed inside */
14         panic("sys__exit returned!\n");
15         break;
16 #endif

```

At the end of the `syscall` function, the trapframe is modified to reflect the result back to the user process: if successful, `tf->tf_v0` holds the return value and `tf->tf_a3` is cleared to 0. If an error occurred, `tf->tf_v0` holds the error code and `tf->tf_a3` is set to 1. Crucially, the program counter `tf->tf_epc` is incremented by 4 bytes (one instruction length) to ensure the CPU does not continuously re-execute the `syscall` instruction upon returning to user mode.

2.5 Implementing File I/O and Memory Protection

The implementations for the file-related system calls are placed in a new file, `kern/syscall/file_syscalls.c`. The objective is to map standard output (file descriptor 1) to the console.

A naive implementation of `sys_write` might simply cast the user's buffer pointer to a C string and iterate over it, calling `putch()` for every character. However, this is a **catastrophic security vulnerability**.

2.5.1 Visualizing Logical Memory and The Danger of `kseg0`

In the MIPS architecture, the 32-bit logical memory space (4GB) is strictly partitioned by the hardware:

0xFFFFFFFF	+-----+	
	kseg2 (Kernel Mapped)	(1 GB)
0xC0000000	+-----+	
	kseg1 (Kernel Uncached)	(512 MB)
0xA0000000	+-----+	
	kseg0 (Kernel Cached)	(512 MB) <-- Direct map to Physical RAM
0x80000000	+=====+	
	kuseg (User Space)	(2 GB) <-- User programs live HERE
0x00000000	+-----+	

The hardware Memory Management Unit (TLB) strictly enforces this boundary: if a user program executing in user mode attempts to read or write to an address $\geq 0x80000000$ (like `kseg0`), the hardware immediately generates a Segmentation Fault and kills the process.

However, when a user program executes the `syscall` instruction, the CPU switches into **kernel mode** to handle the request. If a malicious user program passes a pointer pointing to `kseg0` into `sys_write`, and the kernel blindly dereferences it (e.g., `char c = *user_ptr`), the hardware will **allow** the access because the CPU is currently executing with kernel privileges! The malicious user program has successfully tricked the kernel into reading or overwriting its own internal memory, compromising the entire operating system.

2.5.2 The Solution: `copyin` and `copyout`

! EXAM FOCUS: IMPLEMENTATION REQUIRED

Function: `sys_read(int fd, userptr_t buf, int size)`

Exam Task: *Bug Detection & Fix*

In past exams, a flawed implementation of `sys_read` is provided using `kgets` directly on `buf`. You must identify the security flaw (kernel memory overwrite via malicious pointer) and propose a fix: allocate a temporary kernel buffer, read data into it, and safely transfer to user space using `copyout()`.

To prevent this, the kernel must never trust user-provided pointers. It must rigorously validate that the pointer resides entirely within the safe bounds of `kuseg`. OS161 provides specialized functions for this. Note how they utilize a special `copyfail` fault handler to safely trap bad memory accesses without causing a kernel panic:

```

1  /*
2  * Copy a block of memory of length LEN from user-level address USERSRC
3  * to kernel address DEST. We can use memcpy because it's protected by
4  * the tm_badfaultfunc/copyfail logic.
5  */
6  int copyin(const_userptr_t usersrc, void *dest, size_t len)
7  {
8      int result;
9      size_t stoplen;
10
11     result = copycheck(usersrc, len, &stoplen);
12     if (result) return result;
13
14     if (stoplen != len) return EFAULT;
15
16     curthread->t_machdep.tm_badfaultfunc = copyfail;

```

```
17
18     result = setjmp(curthread->t_machdep.tm_copyjmp);
19     if (result) {
20         curthread->t_machdep.tm_badfaultfunc = NULL;
21         return EFAULT;
22     }
23
24     memcpy(dest, (const void *)usersrc, len);
25
26     curthread->t_machdep.tm_badfaultfunc = NULL;
27     return 0;
28 }
```

And similarly for sending data back to the user:

```
1 /*
2  * Copy a block of memory of length LEN from kernel address SRC to
3  * user-level address USERDEST. We can use memcpy because it's
4  * protected by the tm_badfaultfunc/copyfail logic.
5  */
6 int copyout(const void *src, userptr_t userdest, size_t len)
7 {
8     /* Identical fault protection as copyin, using memcpy to userdest */
9     // ...
10    memcpy((void *)userdest, src, len);
11    // ...
12    return 0;
13 }
```

Exam Insight: System File Table vs User File Table (e.g. Q5 2022-06-20)

The Dual-Table Architecture: When implementing file system calls in OS161, you typically manage two distinct tables:

- **systemFileTable (Global):** Tracks all open files across the *entire* operating system. It holds the `vnode` (the physical file on disk), the global read/write `offset`, and a `refcount` tracking how many processes are currently sharing this file.
- **fileTable (Per-Process):** An array inside `struct proc`. The index of this array is what user-space knows as the **File Descriptor (FD)**. Each entry simply contains a pointer to a specific row in the global `systemFileTable`.

Why this separation? It ensures isolation and security. A process only knows about small integers (FDs like 0, 1, 2) and cannot arbitrarily access files opened by other processes. The kernel uses the FD as an index into the process's `fileTable` to securely resolve the pointer to the global `systemFileTable`.

Shared Entries and lseek: Suppose `P1->fileTable[5]` and `P2->fileTable[8]` both point to the *exact same* `&systemFileTable[20]`.

- **File Descriptors:** P1 is accessing the file via FD 5. P2 is accessing the file via FD 8.
- **Interactivity:** Are their reads independent? **NO!** Because both FDs point to the exact same underlying `systemFileTable` entry, they inherently share the *same* `offset` variable.
- **The effect of lseek:** If P1 executes `lseek(5, ...)`, it modifies the shared `offset` in `systemFileTable[20]`. Therefore, P1's `lseek` will **SEMPRE** (always) affect P2's next `read()`, regardless of whether the seek was `SEEK_SET`, `SEEK_CUR`, or `SEEK_END`.

2.6 Process Termination Constraints: The Suicide Paradox

Process termination is handled in `kern/syscall/proc_syscalls.c`. When a program calls `exit`, its resources must be reclaimed.

A logical first thought would be to simply call `as_destroy()` to free the memory, and then `proc_destroy()` to destroy the process structure. However, a thread cannot call `proc_destroy(curproc)` directly from within `sys_exit`. This is known as the "suicide paradox."

Examine the implementation of `proc_destroy()`:

```

1 void proc_destroy(struct proc *proc)
2 {
3     KASSERT(proc != NULL);
4     KASSERT(proc != kproc);
5
6     /* VFS fields */
7     if (proc->p_cwd) {
8         VOP_DECREF(proc->p_cwd);
9         proc->p_cwd = NULL;
10    }
11
12    /* VM fields */
13    if (proc->p_addrspace) {
14        /* Safely detach and destroy the address space */
15        struct addrspace *as;
```

```

16     if (proc == curproc) {
17         as = proc_setas(NULL);
18         as_deactivate();
19     } else {
20         as = proc->p_addrspace;
21         proc->p_addrspace = NULL;
22     }
23     as_destroy(as);
24 }
25
26     /* THE PARADOX: Asserting the thread array is empty */
27     KASSERT(proc->p_numthreads == 0);
28     spinlock_cleanup(&proc->p_lock);
29
30     proc_end_waitpid(proc);
31     kfree(proc->p_name);
32     kfree(proc);
33 }

```

To fully grasp why `proc_destroy()` is so tricky to call correctly, we must consider what happens when a program terminates via `sys_exit()` under three different scenarios:

Scenario A: The Memory Leak

If your `sys_exit()` implementation simply calls `thread_exit()` without dealing with the process structure, the thread successfully dies. However, nobody ever called `proc_destroy()`. The physical memory allocated to the user program (its Address Space) and the `struct proc` remain allocated in RAM forever. **This causes a severe memory leak.**

Scenario B: The Kernel Panic

To fix the memory leak, your first instinct is to call `proc_destroy(curproc)` right before calling `thread_exit()`. However, `proc_destroy()` contains this exact assertion: `KASSERT(proc->p_numthreads == 0);`. The kernel explicitly asserts that a process **must have zero active threads** before it can be safely destroyed. Since the thread executing `sys_exit()` is still alive inside the process, `p_numthreads` is 1. The assertion fails, resulting in an immediate **Kernel Panic**.

Even if we removed that assertion, it would still be catastrophic: the `proc_destroy` function would deallocate the process data structures, including the thread array and potentially triggering the cleanup of the very kernel stack the thread is currently executing on. The thread would find itself executing code on memory that has just been deallocated.

Scenario C: The Correct Approach (The Zombie State)

! EXAM FOCUS: IMPLEMENTATION REQUIRED

Function: `my_sys_exit(int status)`

Exam Task: *Bug Detection & Context Switch Analysis*

Exams present a buggy `my_sys_exit` calling `thread_exit(curthread)` followed by `proc_destroy(curproc)`. You must identify that calling `proc_destroy` on the currently running process is fatal: it destroys the address space and stack the thread is actively using, causing a Kernel Panic. The thread must mark itself as a zombie and let the parent call `proc_destroy` via `waitpid`.

To safely destroy the process without panicking the kernel, and without losing the exit code before `waitpid()` is called, we split the cleanup duties:

1. **The Child:** In `sys_exit()`, the child destroys its heavy memory (the address space), detaches its thread from the process via `proc_remthread(curthread)` (which drops `p_numthreads` to 0), saves its exit code, marks itself as a **ZOMBIE**, and calls `thread_exit()`. The empty `struct proc` shell remains.
2. **The Parent:** Later, the parent calls `sys_waitpid()`, reads the exit code, and *finally* calls `proc_destroy()` on the zombie child. Since the child's threads are gone (`p_numthreads == 0`), the assertion passes and the memory is safely freed!

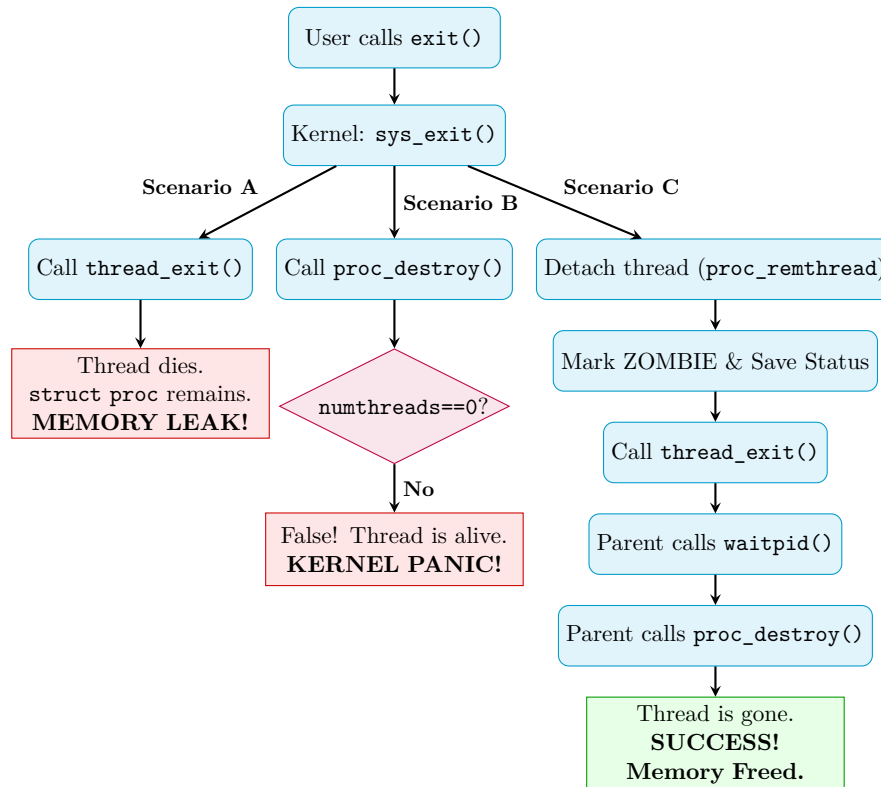


Figure 2.2: Visualization of the three process termination scenarios: Memory Leak, Kernel Panic, and the Correct Zombie Implementation.

2.7 Memory Management: Upgrading DumbVM

OS161's baseline virtual memory manager, `dumbvm.c`, is exceptionally primitive. Physical memory is allocated by a function named `ram_stealmem`.

2.7.1 The Flaw of `ram_stealmem`

! EXAM FOCUS: IMPLEMENTATION REQUIRED

Function: `ram_stealmem(unsigned long npages)`

Exam Task: *Line-by-Line Analysis*

Exams ask whether `ram_stealmem` allocates memory sequentially (Yes) and if it interleaves user/kernel pages (No, it's naive). You may also be asked if a free-list could replace the DumbVM bitmap (Yes, for tracking disjoint free pages).

Take a close look at the original implementation of `ram_stealmem`, which is called heavily during boot and user program execution:

```

1 paddr_t ram_stealmem(unsigned long npages)
2 {
3     size_t size;
4     paddr_t paddr;
5
6     size = npages * PAGE_SIZE;
7
8     if (firstpaddr + size > lastpaddr) {
9         return 0; // Out of memory
10    }
11
12    paddr = firstpaddr;
13    firstpaddr += size;
14
15    return paddr;
16 }

```

`ram_stealmem` allocates memory incrementally. It simply takes the first available physical address (`firstpaddr`), advances the pointer upward by the requested size, and returns the old pointer.

The fatal flaw of `ram_stealmem` is that it implements *allocation without free*. There is no data structure recording which blocks were allocated, nor any mechanism to move `firstpaddr` backward if a middle block is freed. Consequently, repeated execution of user programs inevitably exhausts all physical RAM. Furthermore, because user memory and kernel memory are allocated from the exact same advancing pointer, the physical partitions of the kernel and user processes are permanently interlaced.

To resolve the memory leak, `dumbvm.c` must be upgraded to track the allocation state of individual physical memory frames. There are two primary architectural solutions.

2.7.2 Proposed Solution 1: Bitmap Allocation

This solution introduces two parallel arrays initialized dynamically during `vm_bootstrap` based on the size of the physical RAM:

```

1 static unsigned char *freeRamFrames = NULL; // 0 = free, 1 = occupied
2 static unsigned long *allocSize = NULL;    // Number of contiguous frames
3 static int nRamFrames = 0;                 // Total frames in system

```

1. A **bitmap array** (`freeRamFrames`), where a boolean value indicates whether a specific frame is free or occupied. 2. An **allocSize array**, which records the number of contiguous pages allocated starting at a given frame index. This is strictly required because when `free_kpages` is called with a single starting address, the kernel must know exactly how many subsequent frames belong to that specific allocation to free them correctly.

PROs: * *Simplicity*: Very easy to reason about and implement. * *Rapid Freeing*: Deallocation is $O(1)$ relative to the number of blocks; you just iterate `allocSize` times and flip the bitmap to 0.

CONs: * *Memory Overhead*: The arrays themselves consume physical RAM permanently (one byte plus one unsigned long for every frame in the system). * *Slow Allocation*: Finding a free block of size N requires a linear $O(N)$ scan of the bitmap looking for a contiguous streak of zeros.

2.7.3 Proposed Solution 2: Free-List Allocation

While a bitmap is simple, an alternative implementation uses a **Free-List**—a linked list tracking only the available free intervals. The list nodes store a physical start address and a size.

PROs: * *No Static Overhead*: The list nodes can be embedded directly within the unused physical frames themselves, consuming exactly zero bytes of active system memory. *

****Extremely Fast Best-Fit:**** If the list is ordered by size, finding the tightest-fitting memory block is incredibly fast (just traverse the list until you hit the first block large enough).

CONS: * ****Complex Pointer Manipulation:**** Linking nodes across physical memory requires careful translation between physical and kernel virtual addresses. * ****Slow Merging:**** To combat external fragmentation, when a block is freed, the allocator must search the list to see if the adjacent physical blocks are also free to merge them. If the list is ordered by size, finding physically adjacent blocks requires an $O(N)$ search.

2.7.4 Address Space Duplication (as_copy)

! EXAM FOCUS: IMPLEMENTATION REQUIRED

Function: `as_copy(struct addrspace *old, struct addrspace **ret)`

Exam Task: *Line-by-Line Analysis & CoW Limitations*

The exam asks you to analyze the deep-copy implementation of `as_copy`. You must explain why `PADDR_TO_KVADDR` is strictly required (mapping physical frames into the kernel's virtual segment to perform the `memmove`), and observe that this deep-copy design inherently prevents advanced optimizations like Copy-on-Write (CoW).

When a process forks (a concept explored heavily in later labs), the operating system must perfectly duplicate its entire address space for the child process. This is handled by `as_copy(struct addrspace *old, struct addrspace **ret)`.

A common point of confusion is the function signature: why does `old` have one asterisk while `ret` has two?

- `old (struct addrspace *)` is the source address space. It is passed as a single pointer because it is only being *read*.
- `ret (struct addrspace **)` is the destination. It acts as an **output parameter**. `as_copy` dynamically allocates a new `addrspace` structure in the heap. By passing a double pointer, `as_copy` can modify the original pointer variable in the calling function, pointing it to the newly allocated memory structure.

During duplication, `as_copy` must physically copy the memory contents (code, data, stack) from the parent's physical frames to the child's newly allocated physical frames. It uses the C library function `memmove`.

However, `memmove` is executed by the CPU, which means it requires **virtual addresses**, not physical addresses. The `addrspace` structure stores physical memory addresses (e.g., `as_pbase1`). The MIPS hardware MMU prevents the CPU from directly dereferencing a raw physical address. To bypass this, the kernel uses the `PADDR_TO_KVADDR()` macro. This macro translates the physical address into a `kseg0` virtual address by adding an offset (`0x80000000`). This translated address allows the kernel CPU to legally access and copy the underlying RAM without triggering a TLB fault.

2.7.5 The User Stack Allocation Mechanism (getppages)

Another critical component of `dumbvm` is how the user stack is handled. Unlike code or data segments, which vary in size depending on the program, the user stack in OS161 is assigned a fixed size (`DUMBVM_STACKPAGES`, typically 12 pages) and is always located at the very top of the user logical address space (`USERSTACK`).

Because the logical address is fixed, a common misconception is that the stack does not require physical memory allocation via `ram_stealmem`. This is **false**. While the *logical* layout is fixed, the process still requires actual physical RAM to store its stack variables. In `dumbvm`, the

function `as_define_stack` calls the physical allocator wrapper `getppages()`, which internally relies on `ram_stealmem` (or the upgraded bitmap/free-list) to reserve the necessary physical frames dynamically when the process is created. Crucially, `getppages()` acts as a thread-safe interface between Dumbvm and the RAM. It acquires a spinlock (e.g., `stealmem_lock`) before calling `ram_stealmem` and releases it immediately after. This synchronization guarantees exclusive access to the memory allocation structures, preventing race conditions if multiple threads attempt to allocate memory simultaneously.

2.7.6 MIPS Memory Map: User vs Kernel Space

A fundamental concept in OS161 is understanding how logical (virtual) memory is partitioned between the user application and the kernel. The MIPS architecture divides the 4GB (32-bit) logical address space into distinct segments.

+-----+ 0xFFFFFFFF
kseg2
(Mapped Kernel Space)
+-----+ 0xC0000000
kseg1
(Uncached Kernel RAM)
+-----+ 0xA0000000
kseg0
(Cached Kernel RAM)
+-----+ 0x80000000
kuseg
(User Space Map)
+-----+ 0x00000000

- **kuseg (0x00000000 to 0x7FFFFFFF)**: This 2GB region represents the user space. Every user process believes it owns this entire space. Memory accesses here are translated via the **TLB (Translation Lookaside Buffer)**. In a 32-bit MIPS processor, the TLB is managed partly by hardware and partly by software, and it can hold up to 64 entries. When the hardware MMU cannot find an address translation in the TLB, it triggers a *page fault* exception, which `dumbvm` must handle.
- **kseg0 (0x80000000 to 0x9FFFFFFF)**: This 512MB region is strictly for the kernel. Unlike user space, addresses in `kseg0` are translated *directly by hardware* without the TLB. The physical address is simply the logical address minus 0x80000000 (a process known as relocation). Memory accessed through `kseg0` is **cached** by the CPU.
- **kseg1 (0xA0000000 to 0xBFFFFFFF)**: This 512MB region is also strictly for the kernel and directly translated by hardware. However, unlike `kseg0`, memory accessed through `kseg1` is **uncached** (bypasses the CPU cache).

Exam Insight: I/O Devices and kseg1

Question: *Why are I/O devices mapped to kseg1? Could an I/O device be mapped to kseg0?*

Answer: I/O devices use Memory-Mapped I/O (MMIO), meaning their hardware registers are mapped to specific physical memory addresses. They **cannot** be mapped to kseg0 because kseg0 is **cached**. If the CPU attempts to read a device's status register via kseg0, it might just read an outdated value from the local CPU cache instead of querying the actual hardware device. Similarly, a write command might get trapped in the cache and never reach the peripheral device.

Therefore, I/O devices **must** be mapped to kseg1. Since kseg1 is completely **uncached**, every read and write operation is guaranteed to bypass the CPU cache and travel directly along the bus to the physical hardware device, ensuring accurate and immediate communication.

2.7.7 Logical to Physical Address Translation in OS161

In the primitive dumbvm system, user programs are composed of three primary contiguous regions:

1. **Code Segment (Text):** Defined by `as_vbase1` (Logical Start) and `as_pbase1` (Physical Start).
2. **Data Segment:** Defined by `as_vbase2` and `as_pbase2`.
3. **Stack Segment:** The logical start is always bounded by a fixed ceiling (`USERSTACK = 0x80000000`), growing downwards. The physical start is `as_stackbase`.

To translate a user logical address to a physical address, the kernel uses the following general formula:

$$\text{Physical Address} = \text{Physical Base} + (\text{Logical Address} - \text{Logical Base})$$

However, the stack segment requires a specific calculation step because it grows downward. Its minimum logical base (`VStart`) must be calculated by subtracting its total allocated size from the `USERSTACK` ceiling limit. Once the logical base is found, the standard offset translation formula can be safely applied.

2.8 Laboratory 02: Execute a user program in OS161 (Step-by-Step)

2.8.1 The Goal and The Current Problem

The primary objective of Laboratory 02 is to allow user programs (like `testbin/palin`) to run, print output to the console, and cleanly exit without crashing the kernel.

Why does it crash currently? Out of the box, OS161's system call dispatcher is completely empty. When `palin` attempts to print a character to the screen, it executes the `syscall` instruction with the code for `sys_write`. The kernel traps the exception, jumps to the `syscall()` dispatcher, but finds no `case` for `sys_write`. It hits the default case, returning an "Unknown syscall" error, and fails. Furthermore, when the program naturally finishes execution, it attempts to call `exit()`. Again, the dispatcher has no `case` per `sys__exit`. The kernel panics, and the system halts. Additionally, the original `dumbvm` allocator (`ram_stealmem`) leaks memory, so even if the program could exit, the RAM it used would never be freed.

To solve this, we must:

1. Write the actual logic for printing to the screen and exiting.
2. Wire these functions into the syscall dispatcher.
3. Replace the broken memory allocator to prevent memory leaks.

2.8.2 Step 1: Implementing the System Calls

We need to provide the actual C code that handles printing characters to the console and safely destroying an exiting process.

First, create `kern/syscall/file_syscalls.c` to handle I/O. This utilizes `putch()` and `getch()` for basic console interaction while protecting against `kseg0` attacks:

```

1 #include <types.h>
2 #include <kern/unistd.h>
3 #include <lib.h>
4
5 int sys_write(int fd, userptr_t buf_ptr, size_t size) {
6     int i;
7     char *p = (char *)buf_ptr;
8
9     if (fd != STDOUT_FILENO && fd != STDERR_FILENO) {
10         kprintf("sys_write supported only to stdout\n");
11         return -1;
12     }
13
14     for (i = 0; i < (int)size; i++) {
15         putch(p[i]);
16     }
17     return (int)size;
18 }
19
20 int sys_read(int fd, userptr_t buf_ptr, size_t size) {
21     int i;
22     char *p = (char *)buf_ptr;
23
24     if (fd != STDIN_FILENO) {
25         kprintf("sys_read supported only to stdin\n");
26         return -1;
27     }
28
29     for (i = 0; i < (int)size; i++) {

```

```

30     p[i] = getch();
31     if (p[i] < 0) return i;
32 }
33 return (int)size;
34 }

```

Next, create `kern/syscall/proc_syscalls.c` to handle process termination. This retrieves the current address space, destroys it to free the memory, and safely calls `thread_exit()` to bypass the suicide paradox:

```

1  #include <types.h>
2  #include <proc.h>
3  #include <thread.h>
4  #include <addrspace.h>
5
6  void sys__exit(int status) {
7      /* get address space of current process and destroy */
8      struct addrspace *as = proc_getas();
9      as_destroy(as);
10
11     /* thread exits. proc data structure will be lost */
12     thread_exit();
13
14     panic("thread_exit returned (should not happen)\n");
15     (void) status;
16 }

```

2.8.3 Step 2: Declaring the Syscall Prototypes

Before the kernel can call these new functions in the dispatcher, the C compiler requires function prototypes (blueprints) to ensure the correct number and types of arguments are passed. Open the existing file `kern/include/syscall.h`. Inside the `#if OPT_SYSCALLS` block, add the prototypes for your new functions:

```

1  #if OPT_SYSCALLS
2  int sys_write(int fd, userptr_t buf_ptr, size_t size);
3  int sys_read(int fd, userptr_t buf_ptr, size_t size);
4  void sys__exit(int status);
5  #endif

```

2.8.4 Step 3: Updating the Syscall Dispatcher

When a hardware trap occurs, the kernel must route the numeric syscall code (e.g., 55 for write) to the correct C function we just wrote. Open `kern/arch/mips/syscall/syscall.c` and locate the `switch (callno)` statement inside the `syscall` function. This dispatcher acts as a switchboard. Add the following cases:

```

1  case SYS_write:
2      // Call your safe function, NO direct array access!
3      retval = sys_write((int)tf->tf_a0,
4                          (userptr_t)tf->tf_a1,
5                          (size_t)tf->tf_a2);
6      if (retval < 0) {
7          err = -retval;
8          retval = -1;
9      }
10     break;
11
12 case SYS_read:
13     retval = sys_read((int)tf->tf_a0,
14                       (userptr_t)tf->tf_a1,

```

```

15         (size_t)tf->tf_a2);
16     if (retval < 0) {
17         err = -(retval);
18         retval = -1;
19     }
20     break;
21
22 case SYS__exit:
23     sys__exit((int)tf->tf_a0);
24     // sys__exit never returns
25     break;

```

This explicitly solves the "Unknown syscall" kernel panics.

2.8.5 Step 4: Upgrading the Memory Manager

To fix the memory leak caused by `ram_stealmem`, we must replace the default virtual memory manager. Replace the existing `kern/arch/mips/vm/dumbvm.c` with the upgraded proposed solution.

How does it work differently from `ram_stealmem`? The original `ram_stealmem` simply advanced a pointer to allocate physical memory and had absolutely no mechanism to free it. The new `dumbvm.c` introduces a bitmap allocator using two parallel arrays to track physical memory frames:

```

1 static unsigned char *freeRamFrames = NULL; // 0 = allocated, 1 = free
2 static unsigned long *allocSize = NULL;     // Tracks block sizes

```

Instead of blindly advancing a pointer, the new `getfreeppages()` function scans the `freeRamFrames` array to find a contiguous block of free pages:

```

1 static paddr_t getfreeppages(unsigned long npages) {
2     // ... (scans for npages contiguous free frames)
3     if (found >= 0) {
4         for (i = found; i < found + np; i++) {
5             freeRamFrames[i] = (unsigned char)0; // Mark as occupied
6         }
7         allocSize[found] = np; // Record allocation size
8         addr = (paddr_t) found * PAGE_SIZE;
9     }
10    return addr;
11 }

```

Why does it fix the issues? Because the system now explicitly tracks which frames are free and which are occupied, it allows us to finally implement a working `freeppages()` function. When a process terminates and calls `as_destroy()`, the kernel can now recover the physical RAM:

```

1 static int freeppages(paddr_t addr, unsigned long npages) {
2     long i, first, np = (long)npages;
3     first = addr / PAGE_SIZE;
4     for (i = first; i < first + np; i++) {
5         freeRamFrames[i] = (unsigned char)1; // Mark as free again!
6     }
7     return 1;
8 }

```

By dropping this new `dumbvm.c` implementation into the kernel, the physical memory is properly recycled, fixing the fatal flaw of the original memory manager.

The Ambiguity of Deallocation: Why do we need `allocSize`?

A common question arises when analyzing the upgraded `dumbvm`: *If we have a bitmap (`freeRamFrames`) that tracks which frames are free (1) and which are allocated (0), why do we need a separate*

auxiliary array (`allocSize`) to track the number of frames in each allocation?

The answer lies in the signature of the kernel's fundamental deallocation function:

```
1 void free_kpages(vaddr_t addr);
```

Notice that `free_kpages` accepts only the starting address of the block to be freed; **it does not receive an `npages` parameter**.

If the kernel only had the bitmap, and was told to free the block starting at frame index 5, it would see that frame 5 is 0 (allocated). It checks frame 6, which is also 0. **How does the kernel know if frame 6 is part of the same allocation that started at frame 5, or if it belongs to a completely separate, independent allocation that just happens to be adjacent?** A bitmap segment of [0, 0, 0, 0] could represent:

- One large allocation of 4 pages.
- Four separate allocations of 1 page each.
- One allocation of 2 pages, and two allocations of 1 page.

If it blindly clears contiguous 0s until it hits a 1, it might accidentally deallocate adjacent memory belonging to another process, causing catastrophic memory corruption.

Because the bitmap alone cannot distinguish between "adjacent allocations" and "one large contiguous allocation", we must explicitly store the size `npages` in the `allocSize` array during `alloc_kpages(npages)`. Later, when `free_kpages(addr)` is called, it performs a lookup in the `allocSize` array to determine exactly how many bits to clear in the bitmap!

2.8.6 Step 5: Wiring the New Files into the Build System

Because `file_syscalls.c` and `proc_syscalls.c` are entirely new files, the build system will completely ignore them unless explicitly instructed to compile them. Open `kern/conf/conf.kern`, locate the `defoption syscalls` declaration, and append `optfile syscalls syscall/file_syscalls.c` and `optfile syscalls syscall/proc_syscalls.c`. This tells the compiler to link them into the final kernel binary whenever the `syscalls` feature flag is enabled. Next, ensure your specific configuration file (`kern/conf/DUMBVM`) has this option enabled by verifying it contains the line: `options syscalls`.

2.8.7 Step 6: Reconfigure and Compile

Finally, we must generate the new Makefiles and build the kernel. Open your terminal, navigate to `kern/conf`, and run `./config DUMBVM` to inject the new files into the build tree. Then, navigate to `../compile/DUMBVM` and execute `bmake clean` (to prevent stale object files from causing linker errors), `bmake depend` (to build the header graph), and `bmake` followed by `bmake install` to compile and deploy the final, fully working kernel to the emulator's root directory.

2.8.8 Step 7: Testing the Execution

To verify that everything works correctly, launch the emulator and execute the `palin` test binary.

```
$ cd ~/os161/root
$ sys161 kernel
OS/161 kernel [? for menu]: p testbin/palin
```

Expected Results: If the system calls and memory manager are implemented correctly, the kernel will load the executable and print a string (indicating `sys_write` is functioning).

When the program completes, it will cleanly exit (indicating `sys__exit` works), returning you to the kernel menu prompt *without* crashing. Furthermore, the memory used by `palin` will be recycled by `freeppages`, allowing you to run `p testbin/palin` multiple times consecutively without triggering an "Out of memory" kernel panic.

Past Exam Questions

Group 1: Trapframe vs Switchframe

[Exams 2020-06-27, 2020-09-12, 2021-02-15, 2024-01-22, 2025-01-13] *Quale, tra trapframe e switchframe, viene usato per gestire una system call? Quale viene utilizzato per avviare un processo utente? Quale per la thread_fork? Un trapframe (e lo switchframe) viene allocato nello user stack o nello stack a livello kernel di un processo? (Motivare)*

Answer:

- **Gestione System Call:** Viene utilizzato il **trapframe**. Quando un processo utente esegue l'istruzione `syscall`, si verifica un'eccezione hardware (trap). Il kernel deve salvare lo stato dei registri utente per poterlo ripristinare in seguito, e lo fa all'interno di un trapframe.
- **Avvio Processo Utente:** Viene utilizzato il **trapframe**. La funzione `enter_new_process` crea un trapframe fittizio inserendo i valori iniziali (come il program counter e lo stack pointer dell'utente) e forza il ritorno a user mode eseguendo il "ripristino" di questo trapframe.
- **thread_fork / Context Switch:** Viene utilizzato lo **switchframe**. Quando un thread cede volontariamente la CPU (context switch), il kernel salva lo stato di esecuzione del thread (i registri salvati dal chiamante secondo la convenzione C) in uno switchframe, permettendo al thread di riprendere l'esecuzione kernel esattamente da dove si era interrotto.
- **Allocazione Stack:** Entrambi (trapframe e switchframe) sono allocati **ESCLUSIVAMENTE nello stack a livello kernel** del processo (kernel stack). Il trapframe non può mai risiedere nello user stack per motivi di sicurezza: se il kernel salvasse registri critici o informazioni di stato nello stack utente, un processo malevolo o bacato potrebbe sovrascriverli o corromperli, compromettendo la sicurezza del rientro o del sistema operativo.

Group 2: System Calls, I/O and Memory Protection

[Exams 2020-06-27, 2020-09-12, 2021-02-15] *Un processo utente può eseguire una `write(fd, buf, nb)`, dove `buf` è un indirizzo in `kseg0`?*

Answer: Da un punto di vista dell'invocazione, il processo utente può passare qualsiasi indirizzo numerico come parametro `buf`, incluso un indirizzo appartenente a `kseg0`. Tuttavia, spetta al kernel (nella funzione `sys_write`) rigettare questa richiesta. Poiché la `sys_write` viene eseguita in kernel mode, il processore *non* bloccherà fisicamente l'accesso a `kseg0`. Se il kernel leggesse o scrivesse su quel buffer senza controlli, violerebbe la protezione di memoria, permettendo al processo utente di leggere o corrompere strutture dati critiche del kernel. Per questo motivo, il kernel deve utilizzare funzioni sicure come `copyin` o validare esplicitamente che il puntatore appartenga allo spazio `kuseg`.

[Exams 2020-07-10, 2021-06-09, 2021-09-02, 2023-09-04] *Riferendosi alla system call `sys_read`: 1. E' possibile che la funzione sia richiamata direttamente da un processo user? (Motivare la risposta). 2. La destinazione della `read` può essere nella memoria kernel o deve essere (sempre) nella memoria user? 3. È possibile che il valore ritornato sia diverso dal valore del parametro `size`?*

Answer:

1. **NO, non è possibile.** Un processo utente (come `palin.c`) non può mai chiamare direttamente una funzione kernel in C (come `sys_read`). La separazione degli spazi di indirizzamento e la modalità di esecuzione hardware (user mode) lo impediscono. Il processo utente chiama un wrapper della libreria C standard (es. `read()`), il quale esegue l'istruzione assembly `syscall`, generando un'eccezione hardware che trasferisce il controllo al kernel. Sarà il dispatcher del kernel (`syscall()`) ad invocare la funzione `sys_read`.
2. La destinazione della lettura per una `sys_read` invocata da utente **deve sempre essere nella memoria user** (`kuseg`). Copiare l'input in una zona kernel indicata dal processo utente comprometterebbe la sicurezza del sistema operativo.
3. **Sì, è assolutamente possibile.** La funzione `read` ritorna il numero di byte effettivamente letti. Questo numero può essere inferiore a `size` se si verifica una condizione di End-Of-File (EOF), se c'è un errore di lettura (ritornando un valore negativo), o in caso di dispositivi a stream (come la tastiera o pipe) dove i dati disponibili in quel preciso momento sono inferiori a quelli richiesti.

Group 3: Address Space & Load_Elf

[Exam 2024-07-11 - Q4.B] **Funzioni load_elf** La funzione OS161 `load_elf` predispose lo spazio degli indirizzi. In fasi diverse, chiama diverse funzioni. Associare la funzione corretta: 1. Quale funzione viene chiamata per allocare lo spazio degli indirizzi nella memoria fisica? 2. Quale funzione legge il segmento di codice dal file elf? 3. Quale funzione viene chiamata per impostare gli indirizzi logici e le dimensioni del codice e dei segmenti di dati? 4. Quale funzione legge il segmento di dati dal file elf?

Answer:

1. Allocazione nella memoria fisica: `as_prepare_load()` (questa funzione chiama l'allocatore fisico, come `getppages`, per riservare i frame necessari).
2. Lettura del segmento di codice: `load_segment()` (chiamata per il blocco di codice).
3. Impostazione indirizzi logici e dimensioni: `as_define_region()`.
4. Lettura del segmento di dati: `load_segment()` (esegue la medesima operazione ma sui blocchi dati).

[Exam 2021-07-05] **Funzione as_copy** Perché il parametro `old` ha un solo asterisco mentre `ret` ne ha due? Perché la funzione `memmove` utilizza `PADDR_TO_KVADDR` in questo contesto invece di usare direttamente `as_pbase1`?

Answer: Il parametro `ret` ha due asterischi (`struct addrspace **ret`) perché deve fungere da parametro di output. La funzione `as_copy` deve restituire un puntatore al nuovo address space creato; passando un doppio puntatore, la funzione può allocare la nuova struttura e modificare il puntatore originario nel chiamante per farlo puntare alla nuova area di memoria. Il parametro `old` ha un solo asterisco perché viene solo letto (è l'address space sorgente da copiare).

L'uso di `PADDR_TO_KVADDR` è **obbligatorio** perché la MIPS MMU non permette alla CPU di dereferenziare direttamente indirizzi fisici. `as_pbase1` contiene l'indirizzo fisico in RAM. Per poter leggere o copiare quei dati tramite la CPU (come fa `memmove`), l'indirizzo fisico deve essere tradotto in un indirizzo virtuale accessibile dal kernel. In OS161, la regione di memoria kernel `kseg0` mappa direttamente la RAM fisica aggiungendo un offset; `PADDR_TO_KVADDR` esegue esattamente questa conversione, fornendo un indirizzo virtuale su cui `memmove` può operare senza innescare un TLB fault.

[Exam 2024-09-02 - Q9] **Traduzione Indirizzi** È noto che uno spazio di indirizzamento utente è caratterizzato da `as->as_pbase1`, `as->as_pbase2`, `as->as_vbase1`, `as->as_vbase2`, `as->as_npages1`, `as->as_npages2`, `as->as_stackpbase`, aventi i seguenti valori: `0x200000`, `0x300000`, `0x3000`, `0x7000`, `3`, `4`, `0x400000`. Le pagine in OS161 hanno una dimensione di 4 KB. È anche noto che a uno stack user vengono allocate 18 pagine in `dumbvm`. Dati i seguenti indirizzi logici (potrebbero essere sia user che kernel) convertirli nei relativi indirizzi fisici: `0x8110`, `0x6500`, `0x7FFFE010`, `0x805000B0`.

Answer: Regola generale di traduzione: $\text{Indirizzo Fisico} = \text{Base Fisica} + (\text{Indirizzo Virtuale} - \text{Base Virtuale})$.

- **Processo 1 (Codice e Dati):**

- **0x6500 (Segmento Codice / Pbase1):** L'indirizzo rientra nel range virtuale del codice (da `0x3000` a `0x6FFF`).
 $\text{Offset} = 0x6500 - 0x3000 = 0x3500$.
 $\text{Indirizzo Fisico} = 0x200000 + 0x3500 = 0x203500$.

- **0x8110 (Segmento Dati / Pbase2):** L'indirizzo rientra nel range virtuale dei dati (da 0x7000 a 0xBFFF).

Offset = 0x8110 - 0x7000 = 0x1110.

Indirizzo Fisico = 0x300000 + 0x1110 = 0x301110.

- **Processo 2 (Stack):**

- **0x7FFFE010 (Segmento Stack):** A differenza dei segmenti di codice e dati che crescono verso l'alto (da un indirizzo basso a uno alto), lo stack cresce verso il basso a partire da un tetto massimo fisso. In OS161, questo limite invalicabile è definito dalla costante `USERSTACK = 0x80000000`.

Passo 1: Calcolo della dimensione dello stack. Il testo specifica che sono allocate 18 pagine. Essendo ogni pagina di 4 KB (4096 byte = 0x1000), la dimensione totale è $18 \times 0x1000 = 0x12000$ byte.

Passo 2: Calcolo della Base Virtuale minima (VStart). Sottraendo la dimensione complessiva dal tetto massimo, si ottiene l'indirizzo virtuale più basso valido per lo stack: $0x80000000 - 0x12000 = 0x7FFEE000$.

Passo 3: Calcolo dell'Offset. Si calcola la distanza dell'indirizzo richiesto dalla base virtuale appena trovata: $0x7FFFE010 - 0x7FFEE000 = 0x10010$.

Passo 4: Mappatura Fisica. Si aggiunge l'offset alla base fisica fornita dal testo (`as->as_stackbase`): $0x400000 + 0x10010 = 0x410010$.

- **Indirizzi Kernel:**

- **0x805000B0 (Indirizzo Kernel in kseg0):** Gli indirizzi kernel residenti nel segmento `kseg0` (che va da 0x80000000 a 0x9FFFFFFF) godono di una traduzione diretta via hardware nel processore MIPS. Si ottiene l'indirizzo fisico semplicemente sottraendo l'offset di base del segmento (0x80000000).

Indirizzo Fisico = 0x805000B0 - 0x80000000 = 0x005000B0.

Exam Insight: The Address Classification Trick (e.g. Q15)

A very common exam question asks you to classify an address as a **User Logical Address**, a **Kernel Logical Address**, or a **Physical Address**. The trick to solving this instantly relies on the hardware limit 0x80000000:

- **Addresses $\geq 0x80000000$** (e.g., 0x80803005): These can **ONLY** be *Kernel Logical Addresses* (specifically in `kseg0` or `kseg1`). They cannot be user addresses (they would cause an exception), and they cannot be physical addresses because they far exceed the hardware limits of the simulator's RAM (which is usually just a few MBs).
- **Addresses $< 0x80000000$** (e.g., 0x312010 or 0x532100): These can be **BOTH** *User Logical Addresses* AND *Physical Addresses*. Since they are below the threshold, they are perfectly valid user virtual addresses (`kuseg`). Concurrently, since their numeric value is very small ($\sim 3\text{-}5$ MB), they are perfectly plausible physical addresses within a typical physical RAM size.

Group 4: Process Exit and `proc_destroy`

[Exams 2022-09-08, 2022-10-26, 2023-06-20, 2024-07-11] *Perché il processo che termina (con `sys_exit`) non può chiamare direttamente la `proc_destroy(curproc)`, dopo aver segnalato la fine del processo stesso e prima della `thread_exit`?*

Answer: Questo fenomeno è noto come "suicide paradox". Una chiamata di sistema come `sys_exit` viene eseguita dal kernel per conto del processo utente, utilizzando il **kernel stack** associato al thread corrente (che a sua volta appartiene al `curproc`). Se la `sys_exit` chiamasse `proc_destroy(curproc)`, verrebbero distrutte in memoria tutte le strutture dati del processo, inclusa la memoria stessa che il kernel sta usando per eseguire la chiamata.

Più nello specifico (come evidenziato nelle correzioni d'esame), la struttura `curthread` contiene un puntatore al processo a cui appartiene (`t_proc`). Quando il thread terminerà definitivamente chiamando `thread_exit()`, il codice del kernel andrà a leggere e modificare i contatori del processo tramite quel puntatore (ad esempio, per staccarsi dalla lista dei thread). Se si chiama prima `proc_destroy`, quell'area di memoria viene liberata (`kfree`). Il successivo accesso da parte di `thread_exit()` provocherà un accesso a memoria deallocata (**Use-After-Free**), portando immediatamente a un crash di sistema (kernel panic).

Per questo motivo, il thread in fase di exit deve esclusivamente chiamare `thread_exit()`, delegando la reale deallocazione della struttura processo (`proc_destroy`) al processo padre (solitamente durante l'esecuzione di una `waitpid`), in un momento in cui il thread figlio ha sicuramente cessato ogni esecuzione.

Memory Leaks and Kernel Panics (Missing `as_destroy`):

Cosa succede se, prima di terminare, non viene chiamata la funzione `as_destroy`? La memoria rimarrà allocata, generando un gravissimo **memory leak**. Nello specifico, non viene persa solo la struttura `addrspace`, ma soprattutto i **frame fisici di RAM** (le pagine) precedentemente assegnati al processo. Dato che la RAM fisica del simulatore è strettamente limitata, se i processi terminano continuamente senza liberare le loro pagine fisiche, prima o poi i frame si esauriranno completamente (**Out Of Memory - OOM**). Quando ciò accade, le successive chiamate a `getppages()` falliranno, causando inevitabilmente un **kernel panic** (poiché OS161 base non ha meccanismi di swap o gestione robusta dell'OOM).

Group 5: DumbVM & Memory Allocation

[Exam 2024-09-02 - Q8] Funzione `ram_stealmem` Vero/Falso:

- La funzione `ram_stealmem` alloca in modo incrementale la memoria a partire dal primo indirizzo disponibile dopo aver fatto load del kernel. **VERO**. È esattamente il comportamento di "steal": avanza un puntatore (`firstaddr`) ignorando la deallocazione.
- Quando si utilizza una bitmap per tenere traccia della RAM allocata/libera, è necessario un secondo array che memorizzi le dimensioni delle partizioni allocate per l'allocazione sia della memoria dinamica del kernel che dei processi utente. **VERO**. (Questo è l'array `allocSize`, necessario alla funzione `freeppages` per sapere quanti frame contigui liberare partendo da un indirizzo base).
- Lo stack di un processo utente non viene allocato (in `dumbvm`) chiamando `ram_stealmem`, perché la dimensione dello stack è una costante e lo stack si trova sempre agli stessi indirizzi logici. **FALSO**. Sebbene l'indirizzo logico dello stack sia fisso, la memoria fisica *deve* essere allocata. In `dumbvm` originale, questa allocazione fisica avviene tramite `getppages`, che sotto il cofano chiama proprio `ram_stealmem`.
- Lo schema di allocazione utilizzato da `ram_stealmem` non consentirà l'intercalare partizioni contigue del kernel e dei processi user (in altre parole, le partizioni del kernel e le partizioni user non saranno mai interlacciate). **FALSO**. Poiché `ram_stealmem` è l'unica fonte di memoria fisica per tutto il sistema (nella versione base di `dumbvm`), allocazioni kernel (`kmmalloc`) e allocazioni utente (`as_prepare_load`) attingono dallo stesso serbatoio incrementale in base all'ordine temporale delle richieste, portando inevitabilmente ad un interlacciamento in RAM fisica.

[Exams 2023-01-16, 2023-06-20] Gestione Free-List Si supponga di voler realizzare in `dumbvm` la gestione dei frame liberi mediante free-list, invece che con bitmap. A1) Si dica se la free list può essere una lista non ordinata oppure se deve essere ordinata. (Motivare). A2) Qualora la lista fosse ordinata, andrebbe ordinata in base a indirizzi fisici, logici oppure in base alla dimensione di un intervallo libero? (Motivare).

Answer:

1. **A1:** Teoricamente, la free-list *potrebbe* essere non ordinata, permettendo inserimenti molto veloci ($O(1)$ in testa alla lista) in fase di deallocazione. Tuttavia, mantenere una lista non ordinata rende computazionalmente gravoso ($O(N)$ o peggio) il processo di ispezione e "merging" (fusione) di frame contigui liberati in momenti diversi, portando ad una catastrofica frammentazione esterna. Pertanto, è fortemente raccomandabile e conveniente che sia ordinata.
2. **A2:** A seconda degli obiettivi di ottimizzazione, la lista può essere ordinata in base a criteri diversi (sempre e solo fisici, non logici, poiché stiamo gestendo la RAM reale):
 - **Per Indirizzo Fisico (Crescente):** Ottimale per contrastare la frammentazione esterna. Quando viene liberato un blocco, è facilissimo ispezionare il blocco precedente e successivo nella lista. Se gli indirizzi fisici sono adiacenti, i blocchi vengono fusi in un unico grande intervallo libero.
 - **Per Dimensione (Best-Fit):** Ottimale per la rapidità di allocazione se si vuole implementare la politica Best-Fit. Mantenendo la lista ordinata per dimensione crescente, il primo nodo della lista che soddisfa la dimensione richiesta è matematicamente il blocco più stretto possibile, rendendo la ricerca rapida (mentre la fusione diventa complessa).

Chapter 3

LAB 03: Synchronization in OS161

3.1 Introduction to Synchronization

Modern operating systems are inherently concurrent. Multiple processes or kernel threads often attempt to read and write to the same shared memory structures simultaneously. Without proper orchestration, this concurrency leads to three primary issues:

- **Race Conditions:** The final state of the data depends unpredictably on the exact sequence in which the threads were scheduled.
- **Deadlocks:** Multiple threads block each other indefinitely, each waiting for resources held by another, creating a circular dependency.
- **Resource Starvation:** A thread, although completely valid, is perpetually bypassed by other threads and never gains access to the shared resource.

Synchronization primitives are the tools provided by the operating system to prevent these chaotic scenarios. They enforce mutual exclusion (ensuring only one thread accesses a critical section at a time) and coordination (allowing threads to wait for specific events). In OS161, we explore these tools in increasing levels of abstraction: from raw hardware support to Spinlocks, Wait Channels, Semaphores, Locks, and Condition Variables.

3.2 Hardware Support: Disabling Interrupts vs. Spinlocks

3.2.1 Software Solutions: Peterson's Algorithm and Limitations

Before hardware primitives became standard, mutual exclusion could be implemented entirely in software using algorithms like **Peterson's Solution**. It uses a **turn** variable and a **flag** array to coordinate two threads. However, Peterson's algorithm is unreliable on modern architectures due to:

- **Out-of-Order Execution:** Processors reorder instructions to optimize performance, breaking the strict sequencing Peterson's algorithm relies on.
- **Cache Consistency:** Memory writes by one core might not be immediately visible to another.

To fix this in weakly-ordered systems, **Memory Barriers** are used to force the processor to complete all pending memory operations before proceeding. However, for true scalability and ease of use, hardware atomic instructions are preferred.



Figure 3.1: A classic Deadlock scenario with circular dependency.

3.2.2 The Limitation of Disabling Interrupts

A naïve approach to protecting a critical section is simply to disable interrupts (e.g., using `splhigh()` in OS161) before accessing shared data, and re-enable them (`splx()`) afterward. If interrupts are disabled, the CPU cannot receive the timer interrupt that triggers a context switch, meaning the current thread cannot be preempted.

While this approach works perfectly on a single-core machine, it completely fails on modern multi-core processors. Disabling interrupts is a *local* operation; it only stops the current CPU from switching threads. It does absolutely nothing to prevent a thread running concurrently on CPU 2 from accessing the same shared memory. Thus, multi-core mutual exclusion requires specialized hardware support in the form of atomic instructions.

3.2.3 Spinlocks and the Test-and-Set Instruction

! EXAM FOCUS: IMPLEMENTATION REQUIRED

Function: `spinlock_acquire(struct spinlock *splk)`

Exam Task: *Line-by-Line Analysis & Bug Detection*

Exams ask why `spinlock_acquire` utilizes both `spinlock_data_get` and `spinlock_data_testandset` in its inner loop (to reduce atomic bus-locking traffic when the lock is already held). You may also be asked to analyze a flawed `spinlock_acquire2` to spot missing optimizations or synchronization bugs.

To guarantee mutual exclusion across multiple cores, hardware provides atomic operations like `test-and-set`. A **Spinlock** is the lowest-level synchronization primitive built on this

hardware support.

A spinlock implements *busy-waiting*. If a thread attempts to acquire a spinlock that is already held, it does not go to sleep; instead, it enters a tight loop (it "spins"), continuously probing the lock's status until it becomes available.

```

1 struct spinlock {
2     volatile spinlock_data_t splk_lock; /* The actual hardware lock flag */
3     struct cpu *splk_holder;           /* The CPU currently holding the lock */
4 };

```

Explanation of the Fields:

- **splk_lock:** This is the low-level integer variable that physically represents the lock state (0 for free, non-zero for held). The `volatile` keyword is essential because it ensures the compiler does not aggressively cache this value in a register, forcing it to read the true value from RAM every time (crucial for polling).
- **splk_holder:** A pointer to the CPU currently holding the spinlock. It is primarily used for debugging and catching recursive locks (if the same CPU tries to acquire a spinlock it already holds, the kernel panics to prevent an immediate deadlock).

In OS161, acquiring a spinlock (`spinlock_acquire`) uses two distinct hardware operations in sequence: `spinlock_data_get()` and `spinlock_data_testandset()`.

Exam Insight: Spinlock Implementation (e.g. Q4 2022-09-08)

Exams often ask you to compare two implementations of `spinlock_acquire`:

- **Implementation A (Test-and-Set):** Uses only a `while` loop executing the expensive `spinlock_data_testandset()` operation continuously.
- **Implementation B (Test-and-Test-and-Set):** Uses an outer loop with `spinlock_data_testandset()`, and an inner `while` loop executing the cheaper `spinlock_data_get(&splk->splk_lock)`.

Why is Implementation B (the call to `spinlock_data_get`) strictly necessary? `testandset` is an expensive atomic write operation that locks the hardware memory bus. If multiple CPUs constantly execute `testandset` in a tight loop while waiting for a lock, they will flood the memory bus with write traffic, crippling the performance of the entire system.

By adding the inner `spinlock_data_get()` loop, the CPU performs a cheap, cacheable read. The CPU spins entirely locally on this read operation using its local cache, without touching the memory bus. Only when the read indicates the lock *might* be free does the CPU break the inner loop and attempt the expensive `testandset` to actually grab the lock. This drastically reduces bus contention.

3.2.4 Advanced Hardware Instructions: Compare-and-Swap (CAS)

While `test-and-set` is fundamental, modern architectures also provide **Compare-and-Swap (CAS)**. CAS takes three parameters: a memory location, an expected value, and a new value. It atomically reads the location and, *only if* it matches the expected value, updates it to the new value. This is more powerful than `test-and-set` because it avoids unnecessary memory writes and can handle arbitrary values. CAS is the building block for **Atomic Variables** (like `atomic_int`), allowing counters to be safely incremented without full locks by retrying the CAS

in a loop until it succeeds. It can also be used with an array of waiting flags to create advanced fair locks that prevent starvation.

Because spinlocks employ busy-waiting (wasting CPU cycles), they must only be used to protect extremely short critical sections where the wait time is expected to be minimal.

3.3 The Sleeping Engine: Wait Channels (`wchan`)

For longer waiting periods (e.g., waiting for disk I/O or waiting for a long computation), busy-waiting is unacceptable. The thread must relinquish the CPU and enter the `SLEEP` state. In OS161, this is achieved using **Wait Channels**.

A wait channel is essentially a queue of sleeping threads. The fundamental operations are:

- `wchan_sleep(struct wchan *wc, struct spinlock *lk)`: Puts the current thread to sleep on the wait channel.
- `wchan_wakeone(struct wchan *wc, struct spinlock *lk)`: Awakens exactly one thread (if any are sleeping).
- `wchan_wakeall(struct wchan *wc, struct spinlock *lk)`: Awakens all threads sleeping on the channel.

3.3.1 The Spinlock Sleep Deadlock

A critical and frequently tested design choice is why `wchan_sleep` requires a spinlock as a parameter. When a thread decides it needs to sleep, it must protect the operation of inserting itself into the wait channel's queue. To ensure mutual exclusion on this queue, the thread must hold a spinlock.

However, a fundamental rule of kernel programming is: **You must never go to sleep while holding a spinlock**. If Thread A holds a spinlock and goes to sleep, and Thread B attempts to acquire that same spinlock, Thread B will spin infinitely (busy-waiting) because Thread A can never run to release it. This is a fatal deadlock.

Therefore, the spinlock must be released right as the thread goes to sleep. But if the thread releases the spinlock *before* calling `wchan_sleep`, a race condition occurs: another thread could fire a wakeup signal in that tiny window before the thread actually sleeps, causing the signal to be lost forever.

The solution is to pass the held spinlock *into* `wchan_sleep`. The `wchan_sleep` function performs the context switch and, atomically inside the deep assembly logic of the scheduler, releases the spinlock exactly at the moment the thread transitions into the sleep state. When the thread is later awakened, `wchan_sleep` reacquires the spinlock before returning to the caller.

3.4 Semaphores vs. Locks

Moving up in abstraction, OS161 provides Semaphores and Locks. Both implement *non-busy waiting* (they put threads to sleep using wait channels if the resource is unavailable).

3.4.1 Semaphores

! EXAM FOCUS: IMPLEMENTATION REQUIRED

Function: P(struct semaphore *sem) & V(struct semaphore *sem)

Exam Task: *Line-by-Line Analysis*

You must be able to explain the internal implementation of P() and V(): Why P() uses a while loop instead of an if (Mesa semantics / handling spurious wakeups), the specific role of the internal spinlock protecting the counter, and why wchan_sleep strictly requires the spinlock as a parameter (to release it atomically while sleeping).

Semaphores are one of the most classic synchronization primitives in operating systems, originally invented by Edsger Dijkstra (hence the Dutch names P and V). In OS161, a semaphore is essentially an integer counter protected by a lock, with a queue for threads that are waiting.

1. The Semaphore Structure

Here is the exact definition of a Semaphore in OS161:

```
1 struct semaphore {
2     char *sem_name;           // A string name used mostly for debugging
3     struct wchan *sem_wchan;  // The "Wait Channel" (a sleep queue)
4     struct spinlock sem_lock;  // A low-level hardware lock
5     volatile unsigned sem_count; // The actual integer counter
6 };
```

Explanation of the fields:

- **sem_count:** This is the heart of the semaphore. It represents the number of "resources" currently available. If it is > 0, a thread can proceed immediately. If it is 0, threads must wait.
- **sem_lock:** Because multiple threads might try to read or change **sem_count** at the exact same time, we must protect it. The spinlock ensures that only one CPU can read/modify the counter or the wait channel at a time.
- **sem_wchan:** If **sem_count** is 0, a thread cannot proceed. Instead of endlessly spinning and wasting CPU time, it goes to sleep inside this wait channel.

2. The P() Function (Acquire / Wait / Proberen)

The P operation stands for the Dutch word *Proberen* (to test) or *Passeren* (to pass). You call P() when your thread wants to **acquire** a resource or enter a critical section.

```
1 void P(struct semaphore *sem) {
2     // 1. Acquire the hardware spinlock to ensure exclusive access
3     spinlock_acquire(&sem->sem_lock);
4
5     // 2. Check if resources are available (sem_count > 0)
6     while (sem->sem_count == 0) {
7         // 3. If no resources, go to sleep on the wait channel!
8         // (wchan_sleep safely releases the spinlock while the thread
9         sleeps,
10         // and automatically re-acquires the spinlock when the thread
11         wakes up).
12         wchan_sleep(sem->sem_wchan, &sem->sem_lock);
13     }
14
15     // 4. We now know sem_count > 0. Claim a resource!
16     sem->sem_count--;
```

```

15
16     // 5. Release the hardware spinlock
17     spinlock_release(&sem->sem_lock);
18 }

```

The `P()` function acts as a **blocking** mechanism. If the internal counter is zero, meaning no resources are currently available, the thread will actively yield the CPU and go to sleep. Crucially, the function evaluates the counter within a `while` loop rather than a simple `if` statement. This loop is essential because when a sleeping thread is eventually woken up, another concurrently executing thread might have already snatched the resource in the brief window before the newly awoken thread can reacquire the spinlock. The `while` loop ensures the thread rigorously verifies the condition again before proceeding.

3. The `V()` Function (Release / Signal / Verhogen)

The `V` operation stands for the Dutch word *Verhogen* (to increase) or *Vrijgeven* (to release). You call `V()` when your thread is **done** with a resource and wants to give it back.

```

1 void V(struct semaphore *sem) {
2     // 1. Acquire the hardware spinlock to ensure exclusive access
3     spinlock_acquire(&sem->sem_lock);
4
5     // 2. We are returning a resource, so increase the counter
6     sem->sem_count++;
7
8     // 3. Wake up exactly ONE thread that might be sleeping in the wait
   channel
9     wchan_wakeone(sem->sem_wchan, &sem->sem_lock);
10
11    // 4. Release the hardware spinlock
12    spinlock_release(&sem->sem_lock);
13 }

```

Conversely, the `V()` function is entirely **non-blocking**. A thread calling `V()` never goes to sleep; it simply increments the counter to signal that a resource has been freed. Following this, it calls `wchan_wakeone()` to wake up a blocked thread. Because `V()` only returns a single resource at a time, it logically calls `wchan_wakeone` rather than `wchan_wakeall`; waking up every sleeping thread would be inefficient, as only one of them would actually be able to claim the newly freed resource before the others are forced back to sleep.

3.4.2 Locks and Ownership

A Lock can be conceptualized as a binary semaphore (a semaphore where the counter is restricted to 0 or 1), but with one massive semantic difference: **Ownership**.

Why do we NOT need an owner in a Semaphore?

Semaphores are designed to manage *resource availability* and signal *events*. If a producer thread adds an item to a buffer, it calls `V()` to signal the availability of that item. The thread that consumes the item (calling `P()`) is almost always a completely different thread (the consumer). Semaphores are inherently symmetric communication mechanisms between different threads; therefore, restricting `V()` to the thread that called `P()` would completely break the producer-consumer paradigm.

Why DO we need an owner in a Lock?

Locks are exclusively designed to guarantee **Mutual Exclusion (Mutex)** around a critical section of code. The core principle of a critical section is that only one thread can execute it at a time to prevent data corruption. Therefore:

1. **Safety:** If Thread A acquires a lock to update a shared variable, it is a catastrophic bug if Thread B arbitrarily releases that lock before Thread A is finished. Enforcing ownership ensures that **only** Thread A can relinquish its exclusive access.
2. **Debugging:** If a deadlock occurs, the kernel can inspect the lock's **owner** pointer to identify exactly which thread is hoarding the lock, making debugging infinitely easier.
3. **Priority Inversion:** Advanced schedulers can use ownership to temporarily boost the priority of a low-priority thread holding a lock that a high-priority thread is waiting for (Priority Inheritance Protocol). This is impossible with semaphores because the OS doesn't know who is "supposed" to call `V()`.

To implement a Lock in OS161, one must maintain a pointer to the owning thread. In `lock_release()`, the system must assert that `curthread == lock->owner`. If a different thread attempts to release the lock, the kernel will panic.

Exam Insight: Lock Implementation Details (e.g. Q24-Q26)

Because of the strict ownership requirement, exam questions often probe how a Lock can (or cannot) be built:

- **Can a lock be implemented using a binary semaphore?** Yes, but *not exclusively*. Since a binary semaphore naturally lacks the concept of ownership, it must be paired with an additional element/requirement (a variable tracking the owner thread) to ensure the releasing thread is the same one that acquired it. A binary semaphore without this additional requirement is insufficient.
- **Can a lock be implemented using a wait channel?** Yes. A wait channel provides the exact sleeping queue mechanics needed. Combined with a spinlock and an ownership pointer, it is the standard and most direct way to build a lock in OS161.
- **Can a lock be implemented using a condition variable?** No. Condition variables are high-level primitives used to wait for logical conditions *while already holding a lock*. They cannot be used to build the lock itself.
- **What functions does `lock_acquire` call?** If implemented natively via wait channels, `lock_acquire` will call `wchan_sleep` to suspend the thread if the lock is busy. If the lock is implemented using a binary semaphore, `lock_acquire` will simply call `P()` on that semaphore. It does **not** call `spinlock_data_testandset` (which is for raw spinlocks) or `cv_wait` (which requires a lock).

3.5 Condition Variables and Mesa Semantics

While Locks provide mutual exclusion, **Condition Variables (CVs)** provide event synchronization. A thread uses a CV to wait until a specific logical condition (e.g., "is the buffer full?") becomes true.

Condition Variables are one of the most powerful—but also most misunderstood—synchronization primitives. Unlike Semaphores, a Condition Variable does **not** have an internal counter. It works strictly in tandem with an external Lock.

The Condition Variable Structure

If you look at the design of a CV, it is incredibly minimalistic:

```

1 struct cv {
2     char *cv_name;           // A string name used for debugging
3     struct wchan *cv_wchan;  // The queue where threads wait for the
    condition
4     struct spinlock cv_lock; // Protects the wait channel operations
5 };

```

The essence of a CV lies entirely in the `cv_wchan` field, which acts as a queue for threads to go to sleep when a specific condition is not met yet. Notice what is missing: there is no `count`. Because a CV relies entirely on the state of your program and an **external lock**, there is no need for an internal resource counter. The `cv_lock` spinlock is strictly used to protect the wait channel queue itself during the process of going to sleep.

The `cv_wait()` Function

The `cv_wait` function is called when a thread has checked a logical condition and realizes it cannot proceed. Crucially, a thread must already hold the associated external user lock before making this call.

```

1 void cv_wait(struct cv *cv, struct lock *lock) {
2     // 1. Acquire the CV's internal spinlock to protect the wait channel
    spinlock_acquire(&cv->cv_lock);
3
4
5     // 2. Release the high-level user Lock so other threads can enter the
    critical section
6     lock_release(lock);
7
8     // 3. Go to sleep. wchan_sleep atomically releases the spinlock as we
    sleep.
9     wchan_sleep(cv->cv_wchan, &cv->cv_lock);
10
11    // 4. Reacquire the spinlock (wchan_sleep does this automatically
    before returning)
12    spinlock_release(&cv->cv_lock);
13
14    // 5. Reacquire the high-level user Lock before returning to the caller
15    lock_acquire(lock);
16 }

```

If `cv_wait` did not release the user lock before going to sleep, the system would instantly deadlock, as no other thread would be able to acquire the lock to change the condition the original thread is waiting for. By atomically releasing the lock and sleeping, it allows another thread to enter the critical section, do work, and eventually wake the sleeping thread up. Before returning, `cv_wait` meticulously reacquires the user lock so that the thread can safely resume checking the condition.

The `cv_signal()` Function

When a thread has changed some state in the program and wants to notify exactly one waiting thread that the condition might now be true, it calls `cv_signal`. The thread must hold the associated lock while signaling.

```

1 void cv_signal(struct cv *cv, struct lock *lock) {
2     spinlock_acquire(&cv->cv_lock);
3     wchan_wakeone(cv->cv_wchan, &cv->cv_lock); // Wake ONE sleeping thread
4     spinlock_release(&cv->cv_lock);
5 }

```

It is important to note that `cv_signal` does not release the user lock. It simply peeks into the wait channel and wakes up a single thread. Because the signaling thread still holds the lock, the newly awoken thread will immediately block inside its own `lock_acquire` call (from step

5 of `cv_wait`). The awoken thread will only truly resume execution once the signaling thread finishes its critical section and explicitly releases the user lock.

The `cv_broadcast()` Function

Finally, `cv_broadcast` is used when a state change might allow all waiting threads to proceed simultaneously.

```

1 void cv_broadcast(struct cv *cv, struct lock *lock) {
2     spinlock_acquire(&cv->cv_lock);
3     wchan_wakeall(cv->cv_wchan, &cv->cv_lock); // Wake ALL sleeping threads
4     spinlock_release(&cv->cv_lock);
5 }

```

Instead of waking just one thread, it awakens every single thread in the wait channel. Just like with `cv_signal`, the signaling thread still holds the lock. Therefore, the awoken threads will essentially stampede to re-acquire the lock, but they must do so one-by-one sequentially as the lock is passed around.

3.5.1 The Necessity of the `while` Loop (Mesa Semantics)

A fundamental exam question is why `cv_wait` must always be wrapped in a `while` loop rather than an `if` statement:

```

1 lock_acquire(lock);
2 while (condition == false) {
3     cv_wait(cv, lock);
4 }
5 // Do work
6 lock_release(lock);

```

OS161 implements **Mesa Semantics** (as do almost all modern OSes, unlike Hoare semantics). When Thread A signals a CV, the sleeping Thread B is awakened and moved to the Ready Queue. However, Thread B does *not* execute immediately. Before the scheduler actually grants CPU time to Thread B, a third Thread C could swoop in, acquire the lock, and change the condition back to false.

If Thread B used an `if` statement, it would blindly proceed assuming the condition was still true, leading to corrupted data. By using a `while` loop, Thread B is forced to re-evaluate the condition the moment it actually resumes execution, going back to sleep if a thief (Thread C) altered the state.

3.5.2 CVs vs Wait Channels: The Chain of Calls

While they seem similar, they operate at different levels of abstraction. `cv_wait` is a high-level wrapper that *internally* relies on `wchan_sleep`. The fundamental difference lies in the synchronization primitives they expect:

- `wchan_sleep` expects a **Spinlock** (hardware-level busy-wait).
- `cv_wait` expects a **Lock** (software-level sleeping lock).

Because of this difference, `cv_wait` must execute a complex, asymmetrical chain of lock acquisitions and releases to safely transition from the high-level Lock to the low-level Spinlock before going to sleep.

The visual flow of this chain of calls is illustrated below:

```

[Thread A] calls cv_wait(cv, lock)
|

```

```

|-- 1. spinlock_acquire(&cv->cv_lock)
|      (Protects the CV's internal wait channel)
|
|-- 2. lock_release(lock)
|      (Relinquishes the high-level lock so other threads can enter the critical section)
|
|-- 3. wchan_sleep(cv->cv_wchan, &cv->cv_lock)
|      |-- Context switch begins
|      |-- Atomically releases &cv->cv_lock exactly as the thread sleeps
|
[Thread A is asleep. Later, Thread B calls cv_signal(cv, lock)]
|
|-- 4. wchan_wakeone wakes Thread A.
|-- 5. wchan_sleep reacquires &cv->cv_lock before returning.
|
|-- 6. spinlock_release(&cv->cv_lock)
|
|-- 7. lock_acquire(lock)
|      (Thread A must reacquire the high-level lock before returning to the caller)

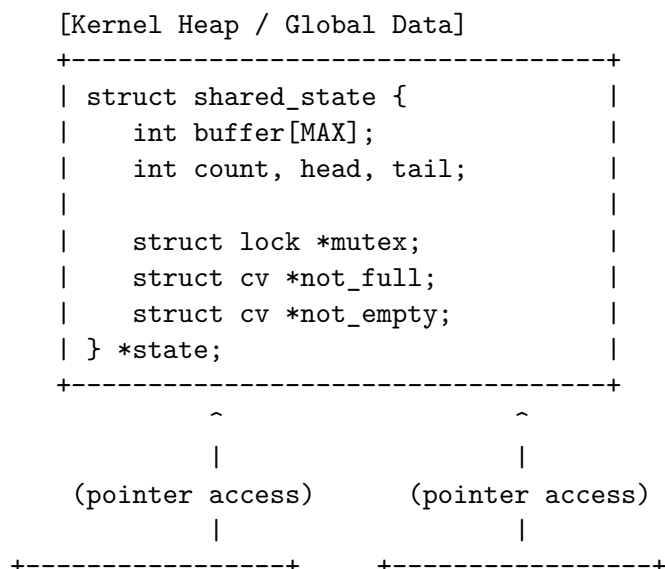
```

3.6 The Producer-Consumer Architecture

To understand how these synchronization primitives (Locks, CVs, and Semaphores) interact in a real operating system scenario, let's examine the classic **Producer-Consumer** problem. This pattern is ubiquitous in OS design (e.g., a keyboard driver producing keystrokes, and a user application consuming them).

3.6.1 Architectural Placement of Shared Data

A critical realization in kernel programming is *where* shared data is stored. If Thread A (Producer) and Thread B (Consumer) need to share a bounded buffer, a Lock, and a CV, **they cannot allocate these structures on their local thread stacks**. Thread stacks are strictly private and disappear when the thread dies. Instead, the shared architecture must be allocated dynamically on the **Kernel Heap** (using `kmalloc`) or as **Global Variables**, ensuring both threads hold pointers to the exact same memory space.



Producer Thread	Consumer Thread
(Stack A)	(Stack B)
+-----+	+-----+

3.6.2 Implementation using Locks and CVs

! EXAM FOCUS: IMPLEMENTATION REQUIRED

Function: `consumerWait(struct prodCons *pc)`

Exam Task: *Implement from Scratch*

Exams often provide a Producer/Consumer struct (with data, size, ready, and busy arrays, plus a lock and CV). You must be able to write the exact C code for `consumerWait` to safely check flags using a `while` loop (Mesa semantics), block using `cv_wait`, and return the index of the buffer to read from, without using busy waiting.

Here is how the producer and consumer safely interact with the shared state using the primitives we discussed:

The Producer must wait if the buffer is full (`count == MAX`):

```

1 void producer(int item) {
2     lock_acquire(state->mutex);
3
4     // Mesa Semantics: always use a while loop!
5     while (state->count == MAX) {
6         cv_wait(state->not_full, state->mutex);
7     }
8
9     // Critical Section: Add item to buffer
10    state->buffer[state->tail] = item;
11    state->tail = (state->tail + 1) % MAX;
12    state->count++;
13
14    // Signal consumers that the buffer is no longer empty
15    cv_signal(state->not_empty, state->mutex);
16
17    lock_release(state->mutex);
18 }
```

The Consumer must wait if the buffer is empty (`count == 0`):

```

1 int consumer() {
2     int item;
3     lock_acquire(state->mutex);
4
5     // Mesa Semantics: always use a while loop!
6     while (state->count == 0) {
7         cv_wait(state->not_empty, state->mutex);
8     }
9
10    // Critical Section: Remove item from buffer
11    item = state->buffer[state->head];
12    state->head = (state->head + 1) % MAX;
13    state->count--;
14
15    // Signal producers that the buffer is no longer full
16    cv_signal(state->not_full, state->mutex);
17
18    lock_release(state->mutex);
19    return item;
20 }
```

This architecture perfectly demonstrates mutual exclusion (via `mutex`) to protect the `count` and buffer indices, combined with conditional synchronization (via `not_full` and `not_empty` CVs) to prevent buffer overflows and underflows.

3.7 Laboratory 03: Step-by-Step Guide

3.7.1 A Note on KASSERT()

Throughout the lab code, you will see frequent uses of `KASSERT(condition)`. This is a kernel-level macro used for strict debugging. If the condition evaluates to false, it triggers a *kernel panic*, printing an error message and immediately halting the operating system. This is crucial for catching fatal invariant violations—such as attempting to release a lock you don’t own, or attempting to sleep inside an interrupt handler—before they cause silent and irreversible data corruption.

3.7.2 The Goal and the Current Problem

Out of the box, OS161 provides a functional Spinlock and Semaphore implementation, but its higher-level synchronization primitives—Locks and Condition Variables (CVs)—are completely empty shells. If you open `kern/thread/synch.c`, you will find functions like `lock_acquire` containing only the comment `// Write this`.

Without these primitives, kernel threads cannot safely share complex data structures without resorting to inefficient busy-waiting (spinlocks). Our objective is to implement Locks and CVs using the **Wait Channel** API, which allows threads to properly sleep and be awakened without wasting CPU cycles.

3.7.3 Step 1: Upgrading the Lock Structure

Before we can write the logic, we must define the memory layout of a Lock. Open `kern/include/synch.h`. A Lock requires three components: a Wait Channel to queue sleeping threads, a Spinlock to protect the Lock’s internal state, and a pointer to track which thread currently owns it.

```

1 struct lock {
2     char *lk_name;
3     // Added for implementation:
4     struct wchan *lk_wchan;    // Queue for sleeping threads
5     struct spinlock lk_lock;    // Protects the wait channel and owner
6     struct thread *lk_owner;    // Tracks ownership for mutual exclusion
7 };

```

3.7.4 Step 2: Implementing Lock Creation and Acquisition

Open `kern/thread/synch.c`. First, update `lock_create()` to initialize the new fields:

```

1 struct lock *lock_create(const char *name) {
2     struct lock *lock = kmalloc(sizeof(*lock));
3     if (lock == NULL) return NULL;
4
5     lock->lk_name = kstrdup(name);
6     if (lock->lk_name == NULL) { kfree(lock); return NULL; }
7
8     lock->lk_wchan = wchan_create(lock->lk_name);
9     if (lock->lk_wchan == NULL) {
10         kfree(lock->lk_name);
11         kfree(lock);
12         return NULL;
13     }
14
15     lock->lk_owner = NULL;
16     spinlock_init(&lock->lk_lock);
17     return lock;
18 }

```

Next, we implement `lock_acquire()`. We use a `while` loop to repeatedly check if the lock is owned. If it is, we sleep on the wait channel. Note how the spinlock is passed directly into `wchan_sleep` to prevent the "lost wakeup" race condition.

```

1 void lock_acquire(struct lock *lock) {
2     KASSERT(lock != NULL);
3     KASSERT(!(lock_do_i_hold(lock))); // Cannot acquire a lock you already hold
4     KASSERT(curthread->t_in_interrupt == false); // Cannot block in interrupts
5
6     spinlock_acquire(&lock->lk_lock);
7     while (lock->lk_owner != NULL) {
8         wchan_sleep(lock->lk_wchan, &lock->lk_lock);
9     }
10    // We woke up, and the owner is NULL. We claim it!
11    KASSERT(lock->lk_owner == NULL);
12    lock->lk_owner = curthread;
13    spinlock_release(&lock->lk_lock);
14 }

```

3.7.5 Step 3: Implementing Lock Release and Ownership Validation

To release the lock, we must verify ownership, clear the owner, and wake up exactly one sleeping thread (if any).

```

1 void lock_release(struct lock *lock) {
2     KASSERT(lock != NULL);
3     KASSERT(lock_do_i_hold(lock)); // Only the owner can release!
4
5     spinlock_acquire(&lock->lk_lock);
6     lock->lk_owner = NULL; // Relinquish ownership
7     wchan_wakeone(lock->lk_wchan, &lock->lk_lock); // Wake up the next thread
8     spinlock_release(&lock->lk_lock);
9 }
10
11 bool lock_do_i_hold(struct lock *lock) {
12     bool res;
13     // We use the spinlock for safe shared data reading
14     spinlock_acquire(&lock->lk_lock);
15     res = (lock->lk_owner == curthread);
16     spinlock_release(&lock->lk_lock);
17     return res;
18 }

```

3.7.6 Step 4: Upgrading the Condition Variable

Condition Variables require similar infrastructure to put threads to sleep. In `kern/include/synch.h`, update the `cv` structure:

```

1 struct cv {
2     char *cv_name;
3     // Added for implementation:
4     struct wchan *cv_wchan;
5     struct spinlock cv_lock;
6 };

```

We must also implement `cv_create` and `cv_destroy` in `synch.c` to initialize and clean up these new fields, identically to how we handled the lock.

```

1 struct cv *cv_create(const char *name) {
2     struct cv *cv = kmalloc(sizeof(*cv));
3     if (cv == NULL) return NULL;
4

```

```

5     cv->cv_name = kstrdup(name);
6     if (cv->cv_name == NULL) {
7         kfree(cv);
8         return NULL;
9     }
10
11    cv->cv_wchan = wchan_create(cv->cv_name);
12    if (cv->cv_wchan == NULL) {
13        kfree(cv->cv_name);
14        kfree(cv);
15        return NULL;
16    }
17
18    spinlock_init(&cv->cv_lock);
19    return cv;
20 }
21
22 void cv_destroy(struct cv *cv) {
23     KASSERT(cv != NULL);
24     spinlock_cleanup(&cv->cv_lock);
25     wchan_destroy(cv->cv_wchan);
26     kfree(cv->cv_name);
27     kfree(cv);
28 }

```

3.7.7 Step 5: Implementing cv_wait and Signals

The `cv_wait` function is the most intricate synchronization puzzle. The thread must release the user's Lock, go to sleep on the CV's wait channel, and reacquire the user's Lock upon waking.

```

1 void cv_wait(struct cv *cv, struct lock *lock) {
2     KASSERT(cv != NULL);
3     KASSERT(lock != NULL);
4     KASSERT(lock_do_i_hold(lock)); // You MUST hold the lock to wait on a CV
5
6     spinlock_acquire(&cv->cv_lock);
7
8     // Atomically release the outer lock right before sleeping
9     lock_release(lock);
10    wchan_sleep(cv->cv_wchan, &cv->cv_lock);
11    spinlock_release(&cv->cv_lock);
12
13    // Reacquire the outer lock upon waking up
14    lock_acquire(lock);
15 }

```

Finally, signaling is straightforward. It simply requires waking threads on the CV's wait channel.

```

1 void cv_signal(struct cv *cv, struct lock *lock) {
2     KASSERT(cv != NULL);
3     KASSERT(lock != NULL);
4     KASSERT(lock_do_i_hold(lock));
5
6     spinlock_acquire(&cv->cv_lock);
7     wchan_wakeone(cv->cv_wchan, &cv->cv_lock);
8     spinlock_release(&cv->cv_lock);
9 }
10
11 void cv_broadcast(struct cv *cv, struct lock *lock) {
12     KASSERT(cv != NULL);
13     KASSERT(lock != NULL);
14     KASSERT(lock_do_i_hold(lock));

```

```

15
16     spinlock_acquire(&cv->cv_lock);
17     wchan_wakeall(cv->cv_wchan, &cv->cv_lock);
18     spinlock_release(&cv->cv_lock);
19 }

```

3.7.8 Step 6: Enabling OPT_SYNCH and Testing

Before compiling the kernel, it is absolutely critical to explicitly enable the synchronization code. As seen in the proposal solution, the code is wrapped in `#if OPT_SYNCH`. If this option is not activated, its default value will be 0, and all your newly written synchronization code will be stripped out by the compiler.

In the OS161 build system, creating a new configuration option requires a two-step process:

Definition and Activation.

1. Definition in `conf.kern`:

First, the kernel configuration system needs to know that this option exists. Open `kern/conf/conf.kern` and declare the new option by appending:

```
defoption synch
```

2. Activation in DUMBVM:

Defining it is not enough; you must turn it on for your specific build. Open your kernel configuration file (e.g., `kern/conf/DUMBVM` or `PROJECT`) and ensure the following line is present to activate it:

```
options synch
```

After performing both steps, reconfigure the kernel to generate the `opt-synch.h` header:

```

$ cd ~/os161/os161-1.11/kern/conf
$ ./config DUMBVM
$ cd ../compile/DUMBVM
$ bmake depend
$ bmake
$ bmake install

```

Finally, navigate to the root directory and launch the simulator to run the synchronization tests (`sy1` for locks, `sy2` for CVs, `sy3` for CVs with locks):

```

$ cd ~/os161/root
$ sys161 kernel "sy1; sy2; sy3; q"

```

Expected Results: The kernel will spawn dozens of concurrent threads that aggressively hammer your Locks and CVs. If your implementations of ownership verification, spinlock management, and Wait Channels are correct, the console will interleave test messages and eventually print "Test passed" for all three tests without panicking, deadlocking, or throwing assertion failures.

Past Exam Questions

Group 1: Multicore Synchronization & Interrupts

[Exams 10/07/2020, 09/06/2021, 02/09/2021] *Si spieghi perché, in un contesto multi-core (sono presenti più CPU) non è possibile realizzare la mutua esclusione semplicemente disabilitando e riabilitando l'interrupt.*

Answer: In un sistema single-core, disabilitare gli interrupt impedisce al timer hardware di scatenare il context switch, garantendo che il thread corrente non venga mai prelazionato (preempted) mentre esegue la sezione critica. Tuttavia, in un sistema multi-core, disabilitare gli interrupt è un'operazione **strettamente locale** alla singola CPU che esegue l'istruzione. Se il Thread A disabilita gli interrupt sulla CPU 1, il Thread B in esecuzione sulla CPU 2 non ne è minimamente influenzato e può tranquillamente accedere alla stessa area di memoria condivisa nello stesso esatto momento. Pertanto, su architetture multi-core, la mutua esclusione richiede obbligatoriamente un supporto hardware esplicito per l'atomicità (come le istruzioni **test-and-set** utilizzate negli spinlock).

Group 2: Wait Channels and Semaphore Internals

[Exams 10/07/2020, 09/06/2021, 02/09/2021] *Riferendosi all'implementazione della primitiva `P()` di un semaforo: 1. A cosa serve lo spinlock? 2. Perché la `P` contiene un ciclo `while` anziché un `if`? 3. Perché la `wchan_sleep` riceve come parametro lo spinlock? 4. E' possibile che la chiamata alla `wchan_wakeone` svegli più di un thread in attesa?*

Answer:

1. Lo spinlock serve a garantire la mutua esclusione sul contatore del semaforo (`sem_count`) e sulla coda del wait channel associato. Siccome le operazioni di test e decremento del contatore devono essere atomiche, lo spinlock evita race conditions.
2. La primitiva `P()` usa un `while` per aderire alla **Mesa Semantics**. Quando un thread viene svegliato da un segnale (`V()`), viene messo in ready queue ma non esegue immediatamente. Prima che ottenga la CPU, un altro thread potrebbe aver già fatto `P()` e decrementato il contatore nuovamente a zero. Il `while` forza il thread risvegliato a ricontrollare se il contatore è effettivamente > 0 , tornando a dormire se necessario.
3. La `wchan_sleep` riceve lo spinlock perché un thread **non può mai addormentarsi possedendo uno spinlock** (causerebbe un deadlock totale, poiché nessun altro thread potrebbe prendere quello spinlock per svegliarlo). Tuttavia, non può rilasciarlo prima di chiamare `sleep`, altrimenti si perderebbe la mutua esclusione creando una finestra per una "lost wakeup" race condition. Passandolo come parametro, la `wchan_sleep` si occupa di rilasciare lo spinlock atomicamente all'interno della procedura di scheduling, esattamente nel momento in cui il thread entra in stato di sleep.
4. **NO**. Per definizione, `wchan_wakeone` estrae e sveglia un singolo thread (il primo) dalla coda di attesa. Per svegliarli tutti si usa `wchan_wakeall`.

Group 3: Condition Variables vs Wait Channels

[Exams 10/07/2020, 09/06/2021, 02/09/2021] *Spiegare brevemente la differenza tra condition variable e wait_channel.*

Answer: Sebbene entrambi servano a gestire code di thread dormienti, operano a livelli di astrazione differenti e si interfacciano con tipi di lock differenti. Il **Wait Channel** è un meccanismo di basso livello utilizzato all'interno del kernel per addormentare un thread. Esegue la protezione della propria coda richiedendo obbligatoriamente il possesso di uno **Spinlock** (busy-waiting lock hardware). La **Condition Variable (CV)** è un costrutto di sincronizzazione di più alto livello per gli utenti. Si associa logicamente ad un **Lock** (sleeping lock software), non a uno spinlock. Quando si chiama `cv_wait`, la CV rilascia il *Lock* associato, utilizza internamente un wait channel (gestendo in autonomia i propri spinlock privati) e riacquisisce il *Lock* al risveglio.

Group 4: Spinlock Internal Implementation

[Exams 08/07/2022, 26/10/2022] Perché `spinlock_acquire` utilizza sia `spinlock_data_get` che `spinlock_data_testandset`, invece di chiamarne solo una? Sulla base del funzionamento, fornire una possibile definizione (linguaggio C) di `struct spinlock`.

Answer: Utilizzare entrambe realizza la fondamentale ottimizzazione nota come **Test-and-Set**. La funzione `spinlock_data_testandset` è un'istruzione hardware atomica di scrittura sulla memoria. Se il processore eseguisse questa istruzione in un ciclo continuo (busy-wait) andrebbe a saturare il bus di sistema con traffico di scrittura, degradando enormemente le prestazioni dell'intero sistema multi-core. Utilizzando `spinlock_data_get()` (una pura lettura), il processore può girare a vuoto leggendo un valore cacheato in locale, senza appesantire il bus di sistema. Solo quando il read indica che la variabile è stata sbloccata (valore zero), il processore esegue l'operazione pesante `testandset` per cercare di aggiudicarsi effettivamente il lock.

Una definizione C di `struct spinlock` in OS161 richiede:

```
1 struct spinlock {  
2     volatile spinlock_data_t lk_lock; // Il flag hardware atomico  
3     struct cpu *lk_holder;           // Puntatore alla CPU che lo possiede  
4 };
```

La variabile deve essere `volatile` per impedire al compilatore di ottimizzarla nei registri, forzando sempre la lettura dalla RAM (o cache coerente) essenziale in un ambiente concorrente. Il puntatore alla CPU serve per il debugging e per identificare violazioni (es. se un thread tenta di prendere uno spinlock che la sua stessa CPU possiede già, causando un deadlock).

Group 5: Producer-Consumer and Memory Allocation

[Exams 20/06/2022, 20/06/2023] *La struttura condivisa può essere posizionata nello stack di thread o dovrebbe essere una variabile globale o altro? Dove dovrebbero essere chiamate `lock_create()` e `cv_create()`? Si fornisca l'implementazione della funzione `consumerWait`.*

Answer: 1. Allocazione della Struttura: La struttura condivisa (il buffer, i flag, i puntatori ai lock e CV) **non può mai essere posizionata nello stack di un thread**. Gli stack dei thread sono entità private e isolate; quando il thread termina, il suo stack viene distrutto, causando "use-after-free" per gli altri thread. Per condividere i dati in modo sicuro, la struttura deve essere allocata nell'area di **memoria heap** (tramite `kmalloc()`) oppure dichiarata come **variabile globale** nel segmento dati del kernel.

2. Inizializzazione Lock e CV: Le funzioni di inizializzazione `lock_create()` e `cv_create()` devono essere chiamate dal thread "padre" o dalla funzione principale di setup **esattamente una volta, prima** che i thread producer e consumer vengano generati (`thread_fork`). Crearle all'interno dei worker thread genererebbe una race condition disastrosa (es. un consumatore potrebbe chiamare `lock_acquire` su un puntatore non ancora inizializzato dal produttore).

3. Implementazione di `consumerWait`: Il consumatore deve attendere finché almeno uno dei flag `ready` diventa 1. Dato che usiamo Condition Variables, dobbiamo aderire alla **Mesa Semantics** (ciclo `while` per il re-check) e proteggere i flag con il lock.

```

1 int consumerWait(struct prodCons *pc) {
2     lock_acquire(pc->pc_lk);
3
4     /* Mesa Semantics: Sleep until AT LEAST ONE buffer is ready */
5     while (pc->ready[0] == 0 && pc->ready[1] == 0) {
6         cv_wait(pc->pc_cv, pc->pc_lk);
7     }
8
9     /* Decide which buffer to consume (prioritize 0) */
10    int index = (pc->ready[0] == 1) ? 0 : 1;
11
12    /* Mark the chosen buffer as busy (in lettura) */
13    pc->busy[index] = 1;
14
15    lock_release(pc->pc_lk);
16
17    return index;
18 }
```

Group 6: Lock Implementations and Deadlocks

[Exam 22/01/2024] *Implementazione Lock: Il lock può essere implementato da: 1) Un semaforo binario, più un elemento/requisito aggiuntivo. 2) Da un semaforo binario, senza alcun altro elemento/requisito. 3) Da un wait channel. 4) Da una condition variable.*

Answer: Le risposte corrette sono la (1) e la (3). 1) Un Lock *può* essere implementato mascherando un Semaforo Binario, ma serve un elemento aggiuntivo: il tracciamento dell'**Owner** (il puntatore al thread che lo ha acquisito). Il lock differisce dal semaforo proprio per il concetto di ownership, garantendo che chi fa release sia lo stesso thread che ha fatto acquire. 3) Un Lock *può* essere implementato internamente ex-novo utilizzando un Wait Channel (per mettere a dormire i thread), associato ad uno spinlock per la mutua esclusione della propria struttura, un flag booleano per lo stato libero/occupato, e il tracciamento dell'Owner.

[Exam 11/07/2024 - Debugging `proc_wait`] *Trovare gli errori in `proc_wait` in cui viene invocato `cv_wait(proc->p_cv, proc->p_lock);` e `lock_acquire(proc->p_lock);`*

Answer: Un errore classico è l'inversione delle operazioni di lock. Un thread **deve possedere il lock prima** di poter chiamare `cv_wait`. Se nel codice buggato si vede una `cv_wait` eseguita *prima* di `lock_acquire()`, o se la `cv_wait` è eseguita ma non protetta in un ciclo `while` (Mesa Semantics), o se il lock non viene mai rilasciato alla fine con `lock_release()`, questi costituiscono errori critici. La `cv_wait` esige il lock perché al suo interno eseguirà dinamicamente un `lock_release()` atomico prima di addormentarsi; se il thread non possiede il lock, la `cv_wait` manderà in crash il kernel per violazione dell'ownership del lock.

Chapter 4

LAB 04: Process Termination and WaitPID

4.1 Introduction: The Interconnected Process Lifecycle

In OS161, a process terminates by invoking the `exit()` standard library function, which triggers the `SYS_exit` system call. Conversely, **`waitpid()` is the specific function in charge of waiting for the termination of a spawned child process.** A parent process calls `waitpid()` to pause its own execution until its child finishes, and then reads the child's exit status.

But *why* must a parent wait for its child?

- **Synchronization:** A parent often cannot safely proceed until the child completes its task. For instance, a shell program must wait for a user's command to finish executing before printing the next prompt, otherwise the terminal output would become chaotically interleaved.
- **Control Flow:** The parent needs to read the child's exit status to know if the execution succeeded or failed, which dictates the parent's next logical step.

This requirement creates a domino effect of interconnected mechanisms:

1. We need the child's exit status to survive after the child dies. This requires the **Zombie State**.
2. We need the parent to be able to locate this Zombie from user-space. This requires assigning a unique integer identifier, the **PID**.
3. The kernel must rapidly map this PID to the Zombie's memory location. This necessitates a global **Process Table**.
4. To avoid confusing parents with stale data, we must assign PIDs safely. This demands a **Circular Buffer** algorithm.

This laboratory explores the intricate dance between `sys_exit` and `sys_waitpid`, demonstrating how these structures depend on one another.

4.2 The Process Table and PID Assignment

To implement `waitpid(pid)`, the kernel must convert an integer PID into a `struct proc *` pointer in $O(1)$ time.

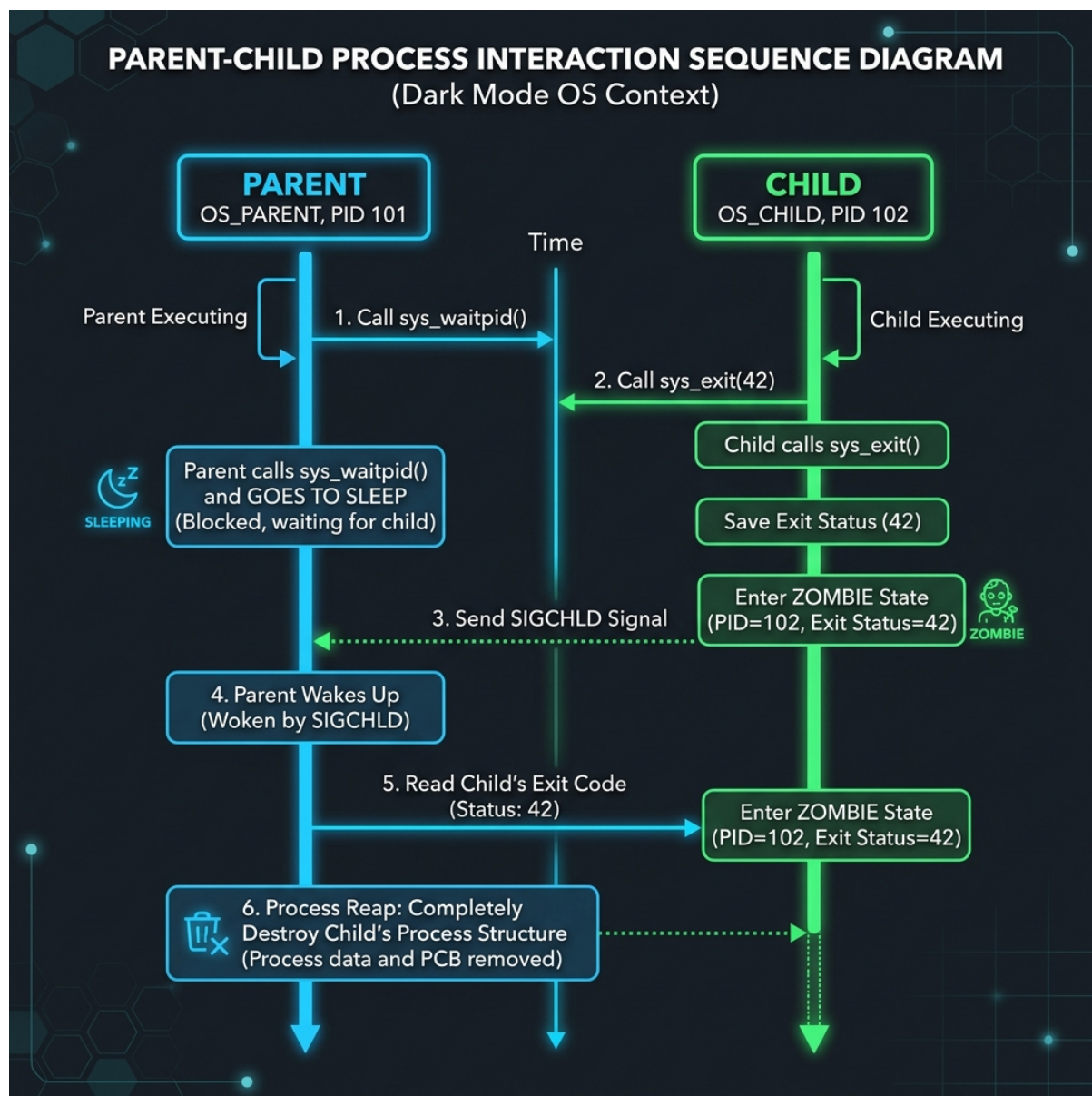


Figure 4.1: The synchronization sequence between a parent calling `waitpid` and a child calling `exit`.

4.2.1 High-Level Structure: The Process Table

We expect the Process Table to be a global array protected by a spinlock. The PID assigned to a process will literally be its index in this array.

```

1 // Expected High-Level Structure
2 struct _processTable {
3     struct proc *proc[MAX_PROC + 1]; // Array of process pointers
4     int last_i;                       // Tracks the last assigned PID
5     struct spinlock lk;               // Mutual exclusion lock
6 } processTable;
7
8 // Expected Conversion Function
9 struct proc *proc_search_pid(pid_t pid) {
10     // 1. Check if PID is within bounds
11     // 2. Acquire processTable.lk
12     // 3. Extract processTable.proc[pid]
13     // 4. Release processTable.lk
14     // 5. Return the extracted pointer
15 }

```

4.2.2 Circular Buffer for PID Assignment

A naive approach to finding a free PID is to always scan the Process Table from index 1 to MAX_PROC and return the very first empty slot. However, this leads to **rapid PID reuse**. If Process 1 dies and is reaped, the very next process spawned will immediately become Process 1.

Why is rapid PID reuse dangerous?

A common misconception is that when a child terminates, the parent is instantly and actively notified. This is not how Unix works. When a child terminates, it becomes a **Zombie**; its exit status sits passively in the process table until the parent explicitly decides to call `waitpid`.

Even after the parent reaps the child (finally freeing the PID), rapid reuse is dangerous due to **stale references** and **asynchronous signals**:

- **Stale References (The Bug):** Imagine a parent process tracks its child's PID (e.g., PID 5). The child dies, and the parent reaps it. PID 5 is now free. Immediately, a completely different user program spawns a new process, which is assigned the newly freed PID 5. If the original parent has a logic bug and mistakenly calls `kill(5)` or `waitpid(5)` *again* (thinking it is still targeting its child), it will accidentally kill or block on an innocent, unrelated process!

To visualize what we expect from the Circular Buffer algorithm to mitigate this:

```

1 // Expected Circular Assignment Logic
2 pid_t assign_pid(struct proc *new_proc) {
3     // 1. Start searching from (last_i + 1)
4     // 2. Loop until we find an index where processTable.proc[i] == NULL
5     // 3. If we hit MAX_PROC, wrap around back to 1
6     // 4. Save new_proc at that index, update last_i, and return the PID
7 }

```

This guarantees that a PID is only reused after MAX_PROC other processes have been spawned, drastically reducing the chance of stale reference bugs.

4.3 The Exit System Call and the Zombie State

When a user program calls `sys_exit(int status)`, the kernel must perform cleanup.

4.3.1 The Zombie State

The Zombie state is the critical mechanism that allows a parent process to reliably capture the exit status of a terminated child. To understand why it is so important, consider the catastrophic bugs that would occur if a dying process directly called `proc_destroy(curproc)` instead of becoming a zombie:

- **Endless Waiting / Lost Status:** If the child completely destroyed itself, its exit status would be instantly erased. Furthermore, if the parent later calls `waitpid()` and attempts to sleep on the child's Condition Variable, but that CV was just wiped from memory by the child's suicide, the parent will either crash or block endlessly waiting for a signal from a thread that no longer exists!
- **The Suicide Paradox:** A dying process executes `sys_exit` using the kernel stack attached to its own thread. If it calls `proc_destroy(curproc)`, it deallocates its own memory structure while still running code on it, causing an immediate kernel panic (a memory fault).

Instead, the process enters the **Zombie State**. It is dead, but its `struct proc` skeleton remains safely preserved in the Process Table, holding the exit status. It is the strict responsibility of the *parent* process (via `waitpid`) to finally read the status and completely obliterate the child.

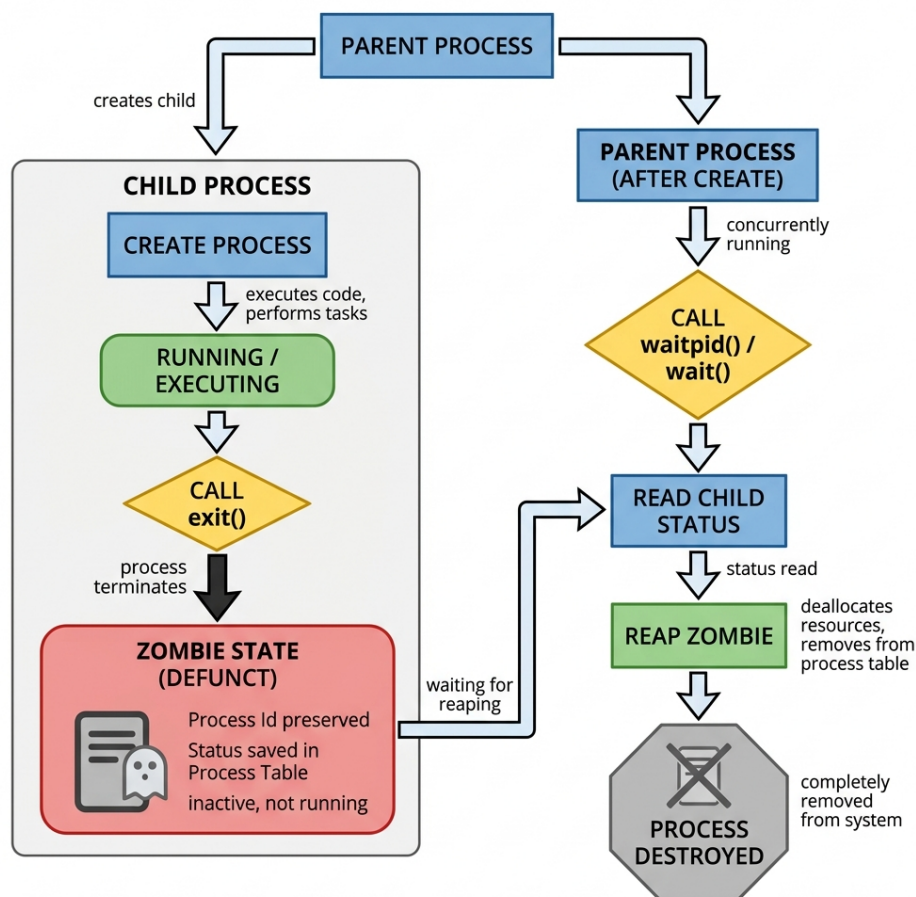


Figure 4.2: Process lifecycle showing the transition into the Zombie state and subsequent reaping by the parent.

```

1 // Expected sys_exit Structure
2 void sys__exit(int status) {
3     // 1. Save the status inside curproc->p_status
4     // 2. Detach the thread from the process
5     // 3. Signal the parent process that we are dying
6     // 4. Call thread_exit() to kill the thread.
7     //     (The proc structure survives as a Zombie!)
8 }

```

4.3.2 The Thread Termination Rule and the Race Condition

A critical rule in OS161 is that a **process cannot be destroyed until all the threads it spawned have ended** (`p_numthreads == 0`).

This creates a subtle but fatal race condition inside `sys_exit`. If `sys_exit` signals the parent *before* explicitly detaching its thread, the parent might wake up and destroy the process while the child thread is still executing its final instructions.

Visualizing the Bug (The Race Condition Sequence):

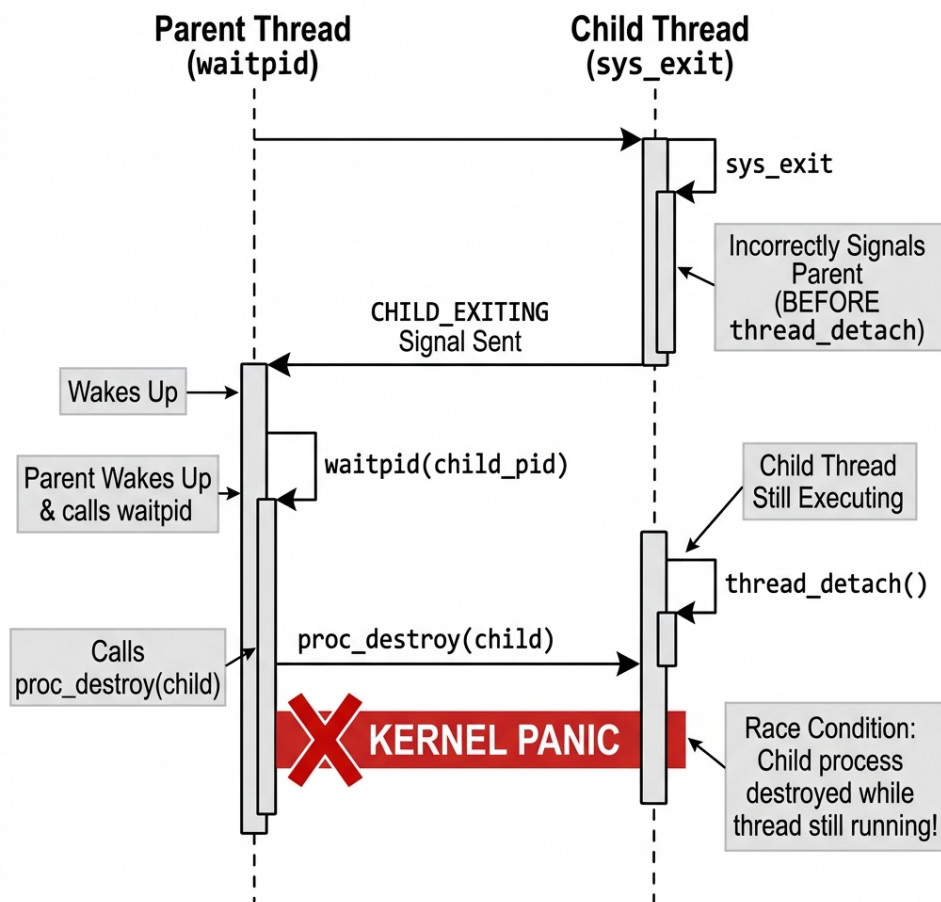


Figure 4.3: Sequence diagram illustrating the race condition in `sys_exit` if the parent is signaled before thread detachment.

CHILD THREAD (<code>sys_exit</code>)	PARENT THREAD (<code>waitpid</code>)
-----	-----
1. <code>curproc->p_status = status;</code>	1. <code>cv_wait(child->p_cv);</code> (Sleeping)
2. <code>cv_signal(curproc->p_cv);</code>	2. WAKES UP!

3. (Context Switch / Preempted)

```
3. proc_destroy(child_proc);
4. KASSERT(child->p_numthreads == 0);
   ~~~~~
   KERNEL PANIC! Assertion fails
   because Child Thread hasn't
   called proc_remthread yet!
```

To fix this, `sys_exit` must **manually and explicitly call `proc_remthread(curthread)`** *before* firing the signal. This detaches the thread, decrementing `p_numthreads` to 0, ensuring the parent's assertion will safely pass even if the child thread is still technically finishing its jump to the graveyard.

4.4 The Wait System Call: `sys_waitpid` vs `proc_wait`

For pedagogical clarity, the waiting logic is split into two distinct layers: User and Kernel.

4.4.1 User Layer: `sys_waitpid`

This is the system call handler. It receives a raw integer `pid` from user space.

```
1 // Expected sys_waitpid Structure
2 int sys_waitpid(pid_t pid, userptr_t statusp) {
3     // 1. Use proc_search_pid(pid) to get the struct proc pointer
4     // 2. Call the internal kernel function: proc_wait(target_proc)
5     // 3. Safely copyout() the returned status to the userptr_t
6 }
```

4.4.2 Kernel Layer: `proc_wait`

This function operates purely on kernel pointers and contains the actual synchronization logic.

Condition Variables vs Semaphores for Synchronization

A parent process waiting for its child **must be evicted** from the CPU (put to sleep); otherwise, it will endlessly consume CPU cycles in a busy-waiting loop, destroying system performance. There are two primary synchronization primitives that can achieve this:

- **Semaphores (`USE_SEMAPHORE_FOR_WAITPID`):** A semaphore initialized to 0 is an elegant solution. The parent simply calls `P(child->p_sem)` to sleep, and the exiting child calls `V(child->p_sem)` to wake the parent.
- **Condition Variables:** If using a CV, a thread cannot simply go to sleep while holding a standard lock (mutex), because the child process will need to acquire that exact same lock in order to send the wake-up signal. If the parent sleeps while holding the lock, the child will deadlock trying to acquire it. Therefore, the `cv_wait` function is strictly required because it atomically puts the parent to sleep *and* releases the lock, allowing the child to safely acquire it, trigger `cv_signal`, and release it.

```
1 // Expected proc_wait Structure
2 int proc_wait(struct proc *child_proc) {
3     // 1. Acquire the child's lock
4     // 2. Sleep on the child's condition variable (cv_wait)
5     // 3. Wake up, read child_proc->p_status
6     // 4. Call proc_destroy(child_proc) to REAP THE ZOMBIE!
7     // 5. Return the status
8 }
```

Exam Insight: Process vs Thread Creation (e.g. Q4 2023-06-20)

thread_switch vs thread_fork: When creating a new executing entity, it is vital to distinguish between these functions. `thread_switch` does **not** create anything; it is simply the low-level assembly routine that performs the context switch (saving and restoring CPU registers). `thread_fork`, instead, allocates and creates the actual kernel thread structure. However, to create a full *process*, both a process structure (via `proc_create`) and a thread (via `thread_fork`) must be created and linked together.

Thread Ownership on Fork: When `thread_fork(const char *name, struct proc *proc, ...)` is called, the newly minted thread belongs to the process passed as the `proc` parameter. It **does not** automatically belong to the calling process! For instance, during a `sys_fork`, the parent process executes the `thread_fork` call, but passes a pointer to the newly created *child* `proc` structure. Thus, the new thread correctly belongs to the child process, not the parent that physically invoked the function.

4.5 Applying Wait Logic to the Kernel Menu

In the baseline OS161 distribution, if you execute a user program from the kernel menu (e.g., `p testbin/add`), the kernel spawns a new thread via `common_prog` and immediately returns to the menu prompt while the program is still running, cluttering the console.

We expect to patch `common_prog` so it waits for the program to finish. But why does `common_prog` directly call `proc_wait` instead of the official `sys_waitpid`?

Because `sys_waitpid` is strictly a **user-space wrapper**. Its entire purpose is to safely transition a user program into the kernel: it validates user privileges, translates an integer PID into a kernel pointer using the Process Table, and securely uses `copyout` to send the integer status back to user memory.

Since `common_prog` is the OS kernel menu, it is already executing with full kernel privileges. It doesn't need to perform user-space security checks, it doesn't need `copyout`, and it doesn't even need to deal with PIDs because it directly possesses the raw `struct proc *` pointer returned by `proc_create_runprogram`. Therefore, it completely bypasses the `sys_waitpid` wrapper and directly invokes the internal kernel engine: `proc_wait(child_proc)`.

```

1 // Expected common_prog Patch
2 int common_prog(int argc, char **argv) {
3     struct proc *proc = proc_create_runprogram(argv[0]);
4     // ... thread_fork occurs here ...
5
6     // NEW: Synchronize the menu by waiting for the child to die!
7     proc_wait(proc);
8     return 0;
9 }

```

4.6 Laboratory 04: Step-by-Step Guide

4.6.1 The Goal and the Current Problem

Currently, OS161 cannot execute user programs cleanly. If you run a program, the kernel does not wait for it to finish. If the program attempts to call `exit()`, the system crashes because the termination logic is incomplete. Our goal is to implement the Process Table, PID management, and the synchronization dance between `sys_exit` and `sys_waitpid` to allow processes to terminate cleanly as Zombies, and for parents to safely wait and read their exit status.

4.6.2 Step 1: Upgrading the Process Structure

Open `kern/include/proc.h`. By default, the `struct proc` only contains fields for memory (`p_addrspace`), threads (`p_numthreads`), and the filesystem (`p_cwd`). There is literally nowhere to store an exit status or a PID. We cannot store the exit status inside the thread structure because the thread is destroyed long before the parent calls `waitpid`. Therefore, we must explicitly add `p_status` and `p_pid`.

Furthermore, we need a way for the parent to safely sleep while waiting for the child to die. We cannot use the existing `struct spinlock p_lock` for this, because threads are strictly forbidden from sleeping while holding a spinlock. Thus, we must introduce a high-level `struct lock *` and a `struct cv *` dedicated solely to this wake-up mechanism.

```

1 struct proc {
2     char *p_name;
3     struct spinlock p_lock;
4     unsigned p_numthreads;
5     struct addrspace *p_addrspace;
6     struct vnode *p_cwd;
7
8     /* Added for Lab 04: Waitpid & Exit Support */
9     int p_status;           /* Exit status returned by sys_exit */
10    pid_t p_pid;            /* Process Identifier */
11
12    struct cv *p_cv;        /* CV to signal the waiting parent */
13    struct lock *p_lock;    /* Lock protecting the CV */
14 };

```

4.6.3 Step 2: Defining the Process Table

Open `kern/proc/proc.c`. While we could theoretically link all processes in a linked list, `sys_waitpid` needs to find a process instantly based on an integer PID. An array of pointers allows $O(1)$ lookups, where the index is the PID itself. We also introduce a dedicated spinlock (`processTable.lk`) because multiple CPUs might attempt to spawn or destroy processes at the exact same nanosecond; without mutual exclusion, the array would become corrupted.

```

1 #define MAX_PROC 100
2 static struct _processTable {
3     int active;
4     struct proc *proc[MAX_PROC + 1]; /* [0] not used. pids are >= 1 */
5     int last_i;                      /* index of last allocated pid */
6     struct spinlock lk;              /* Lock for this table */
7 } processTable;

```

We must initialize this table during kernel boot. Add the following to `proc_bootstrap()`:

```

1 void proc_bootstrap(void) {
2     kproc = proc_create("[kernel]");
3     if (kproc == NULL) panic("proc_create for kproc failed\n");

```

```

4
5     spinlock_init(&processTable.lk);
6     processTable.active = 1;
7 }

```

4.6.4 Step 3: Circular PID Assignment

When `proc_create` is called, we must search the Process Table for a free slot. To implement the circular buffer logic we discussed in the theory section (which prevents the stale reference bugs caused by rapid PID reuse), we add this helper function:

```

1 static void proc_init_waitpid(struct proc *proc, const char *name) {
2     int i;
3     spinlock_acquire(&processTable.lk);
4     i = processTable.last_i + 1;
5     proc->p_pid = 0;
6
7     if (i > MAX_PROC) i = 1;
8     while (i != processTable.last_i) {
9         if (processTable.proc[i] == NULL) {
10             processTable.proc[i] = proc;
11             processTable.last_i = i;
12             proc->p_pid = i;
13             break;
14         }
15         i++;
16         if (i > MAX_PROC) i = 1;
17     }
18     spinlock_release(&processTable.lk);
19
20     if (proc->p_pid == 0) panic("too many processes. proc table is full\n");
21
22     proc->p_status = 0;
23     proc->p_cv = cv_create(name);
24     proc->p_lock = lock_create(name);
25 }

```

Ensure this function is called at the end of `proc_create()`.

4.6.5 Step 4: Implementing `sys_exit`

Open `kern/syscall/proc_syscalls.c`. When a process exits, it must perform four critical steps motivated by the Zombie architecture: 1. It must mask the status with `status & 0xff`. (OS161, like POSIX, only uses the lower 8 bits for the actual exit code). 2. It must detach its thread **before** signaling the parent, otherwise it triggers the fatal race condition panic we visualized earlier. 3. It must wake the parent using `cv_signal`. 4. It must call `thread_exit()` to kill the thread, leaving the `proc` intact as a Zombie.

```

1 void sys__exit(int status) {
2     struct proc *p = curproc;
3
4     /* Save just the lower 8 bits returned */
5     p->p_status = status & 0xff;
6
7     /* EXTREMELY IMPORTANT: Detach thread before signaling parent! */
8     proc_remthread(curthread);
9
10    /* Wake up the parent waiting in waitpid */
11    lock_acquire(p->p_lock);
12    cv_signal(p->p_cv);
13    lock_release(p->p_lock);

```

```

14
15  /* The thread dies, but the proc structure remains as a Zombie */
16  thread_exit();
17  panic("thread_exit returned (should not happen)\n");
18 }

```

4.6.6 Step 5: Implementing `proc_wait`

! EXAM FOCUS: IMPLEMENTATION REQUIRED

Function: `proc_wait(struct proc *proc)`

Exam Task: *Bug Detection*

Exams present a buggy version of `proc_wait` containing critical synchronization errors. You must be able to spot: 1) calling `cv_wait` *before* `lock_acquire`, 2) returning an error status without releasing the lock (deadlock), and 3) wrongly destroying the parent `curproc` instead of the child `proc`.

The parent process waits for the child by sleeping on its CV. Once awakened, the parent must read the exit status **before** calling `proc_destroy`. This ordering is strictly motivated by memory safety: if the parent called `proc_destroy` first, the Zombie would be obliterated, and reading `p_status` would cause a use-after-free memory fault! Back in `proc.c`:

```

1  int proc_wait(struct proc *proc) {
2      int return_status;
3      KASSERT(proc != NULL);
4      KASSERT(proc != curproc);
5
6      lock_acquire(proc->p_lock);
7      cv_wait(proc->p_cv, proc->p_lock);
8      lock_release(proc->p_lock);
9
10     return_status = proc->p_status;
11     proc_destroy(proc);
12     return return_status;
13 }

```

4.6.7 Step 6: Table Lookup and Cleanup Functions

! EXAM FOCUS: IMPLEMENTATION REQUIRED

Function: `proc_to_pid & proc_from_pid`

Exam Task: *Implement from Scratch*

You must know how to build the mapping between a `pid_t` and a `struct proc *`. The exam explicitly asks you to realize these conversion functions and explain the global data structure (the process table/array) and the necessary locking synchronization required to make them thread-safe.

To complete the architecture, we must implement two critical helper functions in `proc.c`.

First, `proc_search_pid` is used by `sys_waitpid` to safely extract a process pointer from the table:

```

1  struct proc *proc_search_pid(pid_t pid) {
2      struct proc *p;
3      KASSERT(pid >= 0 && pid < MAX_PROC);
4      spinlock_acquire(&processTable.lk);
5      p = processTable.proc[pid];
6      spinlock_release(&processTable.lk);

```

```

7  return p;
8  }

```

Second, when `proc_destroy` finally obliterates the Zombie, it must free the PID slot so it can be reused later. It must also destroy the synchronization primitives we added. We isolate this in `proc_end_waitpid`:

```

1  static void proc_end_waitpid(struct proc *proc) {
2      int i;
3      spinlock_acquire(&processTable.lk);
4      i = proc->p_pid;
5      KASSERT(i > 0 && i <= MAX_PROC);
6      processTable.proc[i] = NULL;
7      spinlock_release(&processTable.lk);
8
9      cv_destroy(proc->p_cv);
10     lock_destroy(proc->p_lock);
11 }

```

Important: You must explicitly add a call to `proc_end_waitpid(proc)`; at the very end of the `proc_destroy()` function!

4.6.8 Step 7: Enabling the WAITPID Option

If you wrap your implementation inside the `#if OPT_WAITPID` macro, the compiler will completely ignore your code unless you explicitly configure the kernel to activate it.

To do this, you must modify two files:

1. Open `kern/conf/conf.kern`. This tells the configuration system that the option exists. Find the other `defoption` lines and add:

```
defoption waitpid
```

2. Open your specific kernel configuration file, e.g., `kern/conf/DUMBVM`. This explicitly turns the option on for the build. Add:

```
options waitpid
```

When you subsequently run `./config DUMBVM`, the build system will automatically generate the header `opt-waitpid.h` containing `#define OPT_WAITPID 1`, effectively activating all your new code.

4.6.9 Step 8: Testing the Execution

Compile your kernel and run a simple user program like `testbin/add`:

```

$ sys161 kernel
OS/161 kernel [? for menu]: p testbin/add

```

Expected Results: The program will execute, print the result of the addition, and then cleanly return to the OS/161 menu. Behind the scenes, the child executed `sys_exit`, became a Zombie, and the parent (the kernel menu thread) executed `proc_wait`, reaped the Zombie, and resumed operation without panicking.

4.7 Global Architecture: The Process Termination Call Graph

To provide absolute clarity on the overall "dance" between process termination and synchronization, the following call graph summarizes the entire architecture.

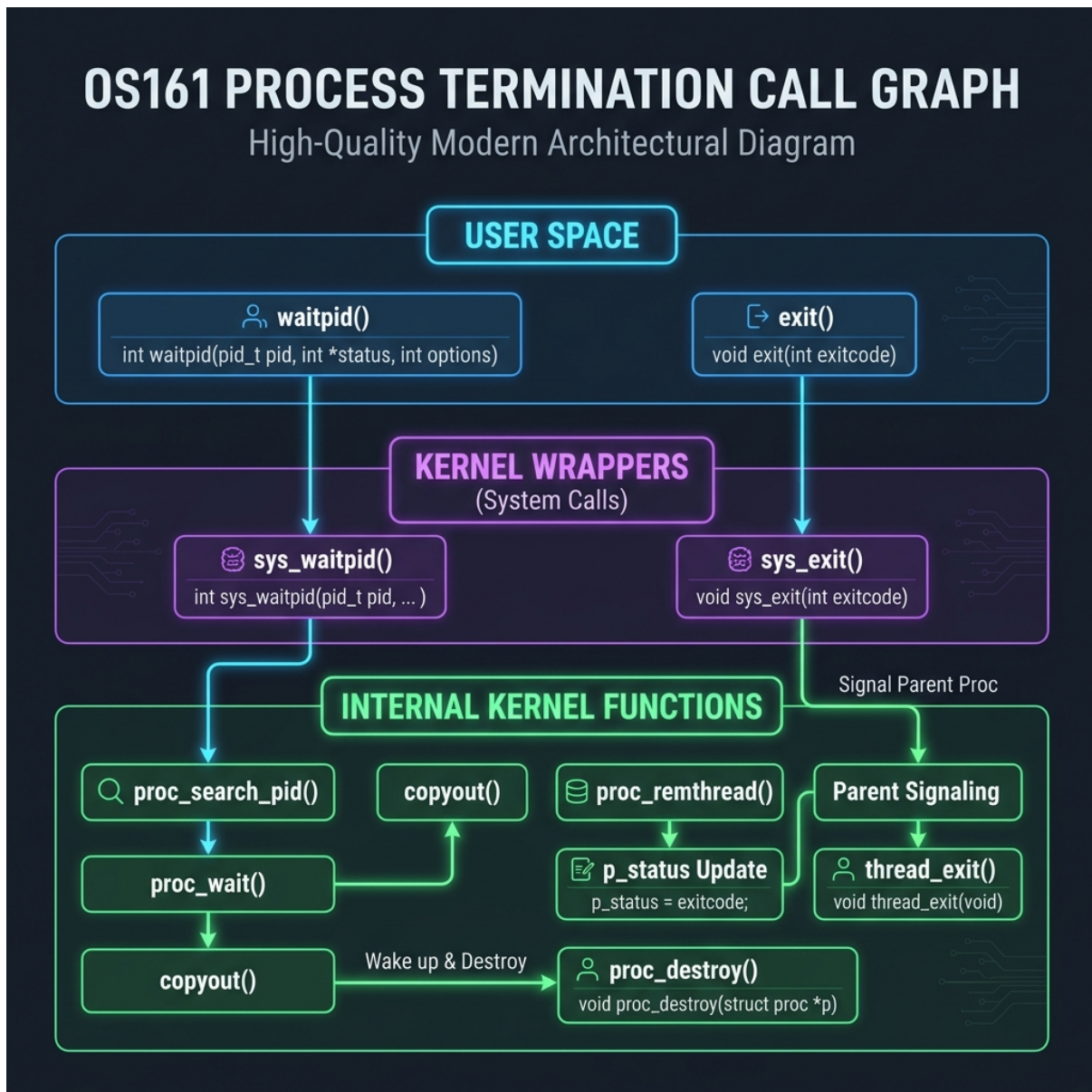


Figure 4.4: The architectural call graph for Process Termination and WaitPID in OS161.

The architecture is divided into three distinct layers:

1. **User Space:** The user applications call the standard C library functions `waitpid()` and `exit()`. These trigger hardware traps to transition securely into the kernel.
2. **Kernel Wrappers (`sys_waitpid` & `sys_exit`):** These act as the boundary layer. They validate user pointers, translate integer PIDs into actual kernel `struct proc *` pointers using `proc_search_pid()`, and manage the safe transfer of data between user and kernel space (via `copyout`).
3. **Internal Kernel Functions:** This is where the core logic executes.

- `proc_wait()` executes the blocking synchronization (via Semaphore or Condition Variable), pausing the parent.
- `sys_exit` performs the teardown: it detaches the thread (`proc_remthread`), updates the zombie status (`p_status = exitcode`), signals the parent to wake up, and finally calls `thread_exit()` to kill the executing thread.
- Finally, the awakened parent finishes `proc_wait()`, returns the exit status via `copyout()`, and completely obliterates the child's data structures by calling `proc_destroy()`.

4.8 The Open File Table (Preview)

! EXAM FOCUS: IMPLEMENTATION REQUIRED

Function: `struct openfile`

Exam Task: *Analysis & Process Interaction*

Although file descriptors are covered extensively later, OS161 exams specifically ask about the interaction between the extbfglobal system-wide `systemFileTable` and the extbfprocess-local `fileTable` array inside `struct proc`. You must understand what happens when two processes have pointers to the exact same global `struct openfile` (e.g., they share the file offset during an `lseek`).

Past Exam Questions

Group 1: The Exit System Call and Zombies

[Exams 2021-07-05, 2022-09-08, 2022-10-26, 2024-07-11] *Perché il processo che termina (con `sys_exit`) non può chiamare direttamente la `proc_destroy(curproc)`, dopo aver segnalato la fine del processo stesso e prima della `thread_exit`?*

Answer: Questo problema racchiude due concetti fondamentali. Primo: Il paradosso del suicidio. Il processo in fase di terminazione sta eseguendo la `sys_exit` avvalendosi del kernel stack associato al proprio thread. Se chiamasse `proc_destroy(curproc)`, deallocherebbe dalla memoria l'intera struttura del processo, compresi gli array dei thread e potenzialmente il proprio stack, finendo per eseguire istruzioni su memoria corrotta e causando un kernel panic. Secondo: Lo stato Zombie. Se il processo si autodistruggesse, il suo `exit status` andrebbe perduto. Il processo deve rimanere in uno stato "Zombie" (struttura dati integra ma nessuna esecuzione) affinché il processo padre possa leggere lo status tramite una futura chiamata a `waitpid`. Spetta esclusivamente al padre invocare la `proc_destroy` per mietere lo zombie.

[Exam 2021-07-05] **Implementazione `my_sys_exit`** *Si definisca il prototipo e la chiamata in `syscall()`:*

```
1 int my_sys_exit(int status);
2
3 void syscall(struct trapframe *tf) {
4     /* ... */
5     case SYS_exit:
6         err = my_sys_exit((int)tf->tf_a0);
7         break;
8 }
```

Nel frammento incompleto, da dove proviene `status` e a cosa va assegnato? Quali errori ci sono?

```
1 my_sys_exit (int status) {
2     curproc->p_status = status;
3     thread_exit(curthread);
4     proc_destroy(curproc); // ERRORE CRITICO
5     as_destroy(curproc->p_addrspace); // ERRORE (Dead Code)
6 }
```

Il valore `status` proviene dal registro `a0` (il primo parametro passato dall'utente) e deve essere salvato in un campo apposito della `struct proc` (es. `curproc->p_status`) affinché il padre possa leggerlo in futuro. Gli errori successivi sono gravissimi: 1. La chiamata `thread_exit` non ritorna **mai** (chiude il thread in via definitiva). Pertanto, le istruzioni successive sono "dead code" inesequibile. 2. Anche scambiando l'ordine, chiamare `proc_destroy(curproc)`

viola la semantica degli zombie e causa crash, poiché distrugge il processo in esecuzione. 3. La distruzione dell'address space (`as_destroy`) andrebbe fatta *prima* della `thread_exit`.

Group 2: The Process Table and PIDs

[Exams 2022-09-08, 2022-10-26] *Si realizzino le funzioni per passare da puntatore a processo a pid e viceversa. Si spieghi a parole cosa siano le strutture dati necessarie per effettuare le conversioni.*

Answer: Per effettuare queste conversioni con complessità $O(1)$, è necessaria una **Process Table**. Si tratta di un array statico globale di puntatori a processo (`struct proc *process_table[MAX_PROC + 1]`). Il PID associato a un processo corrisponde esattamente all'indice dell'array in cui è memorizzato il suo puntatore. Serve inoltre uno `spinlock` globale per garantire la mutua esclusione durante la lettura o la scrittura della tabella.

Versione 2022-09-08 (Ritorno diretto):

```

1 pid_t proc_to_pid(struct proc *p) {
2     // Il PID e' pre-salvato all'interno del processo durante la sua creazione
3     return p->p_pid;
4 }
5
6 struct proc *proc_from_pid(pid_t pid) {
7     struct proc *p = NULL;
8     if (pid > 0 && pid <= MAX_PROC) {
9         spinlock_acquire(&process_table_lock);
10        p = process_table[pid];
11        spinlock_release(&process_table_lock);
12    }
13    return p;
14 }
```

Versione 2022-10-26 (Ritorno by-pointer e codice errore):

```

1 int proc_to_pid(struct proc *p, pid_t *ppid) {
2     if (p == NULL || ppid == NULL) return EINVAL;
3     *ppid = p->p_pid;
4     return 0;
5 }
6
7 int proc_from_pid(pid_t pid, struct proc **pproc) {
8     if (pid <= 0 || pid > MAX_PROC || pproc == NULL) return EINVAL;
9
10    spinlock_acquire(&process_table_lock);
11    struct proc *p = process_table[pid];
12    spinlock_release(&process_table_lock);
13
14    if (p == NULL) return ESRCH; // Process non trovato
15    *pproc = p;
16    return 0;
17 }
```

Group 3: Implementing `sys_waitpid`

[Exam 2022-01-17] Si realizzi una funzione `sys_waitpid`, assumendo già realizzata `proc_wait(struct proc *proc)`.

Answer: La funzione `sys_waitpid` agisce da collante (wrapper) tra lo spazio utente (che parla per PIDs) e lo spazio kernel (che ragiona per puntatori). Deve: estrarre il puntatore dalla tabella usando il PID, gestire il caso di PID invalido, chiamare `proc_wait`, e infine copiare in modo sicuro il codice di uscita nello spazio utente usando `copyout`.

```
1 int sys_waitpid(pid_t pid, userptr_t statusp) {
2     struct proc *target_proc;
3     int status;
4     int err;
5
6     // 1. Traduzione da PID a struct proc
7     target_proc = proc_from_pid(pid); // usa l'array process_table
8     if (target_proc == NULL) {
9         return ESRCH; // Processo non trovato
10    }
11
12    // 2. Chiamata alla logica di sincronizzazione kernel interna
13    status = proc_wait(target_proc);
14
15    // 3. Ritorno dello stato all'utente in modo sicuro
16    if (statusp != NULL) {
17        err = copyout(&status, statusp, sizeof(int));
18        if (err) {
19            return err; // E.g., EFAULT se statusp punta ad area kseg0
20        }
21    }
22
23    // Ritorna il PID del processo terminato in caso di successo
24    return pid;
25 }
```

Group 4: Debugging `proc_wait`

[Exam 2024-09-02 - Q11] Debugging `proc_wait` *Trovare i 3 errori nella funzione `proc_wait` fornita e spiegarli.*

Answer: Ecco il codice originale con gli errori:

```
1 cv_wait(proc->p_cv, proc->p_lock); // ERRORE 1: Mancanza di lock
2 lock_acquire(proc->p_lock);
3 return_status = proc->p_status;
4 /* ... */
5 proc_destroy(curproc); // ERRORE 2 & 3: Oggetto sbagliato e Mesa Semantics
```

Errore 1: Lock assente prima della `cv_wait`. La `cv_wait` richiede OBBLIGATORIAMENTE che il lock passato come parametro sia già posseduto dal thread chiamante, poiché internamente eseguirà il rilascio di tale lock prima di addormentarsi. Nel codice, `cv_wait` viene chiamata *prima* di `lock_acquire`. Questo causerà un kernel panic. **Errore 2: Assenza del ciclo While (Mesa Semantics).** La `cv_wait` non è protetta da un ciclo `while (!proc->p_exited)`. Poiché OS161 usa la semantica Mesa, il risveglio da una CV non garantisce che la condizione sia ancora vera. Senza un ciclo di guardia e una flag booleana esplicita per marcare la terminazione, il thread potrebbe procedere prematuramente, leggendo uno status "spazzatura". **Errore 3: Distruzione del processo sbagliato.** La chiamata `proc_destroy(curproc)` è devastante. Si sta chiedendo al kernel di distruggere il processo chiamante (es. il padre che sta eseguendo la `waitpid`), commettendo suicidio a metà operazione! Il processo corretto da distruggere (mietere lo zombie) è `proc_destroy(proc)`, ovvero il processo figlio ormai defunto.

Group 5: sys_getpid properties

! EXAM FOCUS: IMPLEMENTATION REQUIRED**Function:** sys_getpid()**Exam Task:** *Analysis*

Exams test your knowledge of `sys_getpid()`: it takes no arguments, it is not internally required by `waitpid`, and it cannot be called directly by a user process (it must trap to the kernel dispatcher).

[Exam 2024-09-02 - Q10] `sys_getpid` Vero/Falso Considerare la funzione `sys_getpid` (prototipo sotto): `pid_t sys_getpid(void);`. Per ciascuna delle seguenti frasi, rispondere vero/falso:

Answer:

- La funzione è necessaria per implementare `sys_waitpid`, perché è necessario il pid del processo in attesa. **FALSO**. A `sys_waitpid` non interessa il PID del processo in attesa (il padre chiamante), ma riceve in input il PID del processo figlio da attendere.
- La funzione è necessaria per implementare la chiamata di sistema `getpid`, e sarà chiamata dal kernel, nella funzione `syscall`. **VERO**. È esattamente il ruolo dell'handler kernel della system call.
- La funzione è necessaria per implementare la chiamata di sistema `getpid` e verrà chiamata direttamente da un processo utente. **FALSO**. Un processo utente non può chiamare direttamente codice C del kernel; effettua una trap hardware chiamando il wrapper in assembly.
- La funzione restituisce il pid del processo corrente. **VERO**. Ritorna semplicemente `curproc->p_pid`.
- La funzione viene chiamata per ottenere il pid di un altro processo (quello di cui attendere la terminazione). **FALSO**. Restituisce il proprio, non quello altrui.
- La funzione viene chiamata da `sys_exit`, in modo da conoscere il pid del processo a cui inviare una segnalazione. **FALSO**. In `sys_exit`, la segnalazione avviene tramite strutture sincronizzative registrate internamente nei puntatori del kernel, non è basata sui PID.